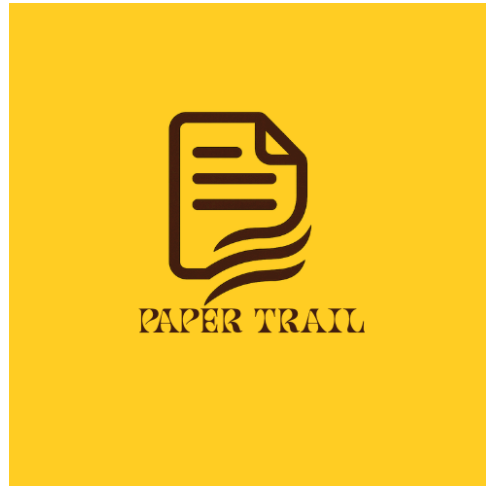




Paper Trail

Critical Design Review Document

Dr. Zihan Wu

Ben Yandell, Robert Kulow, Anthony Veilleux, Arius Ahmad, Vasu Patel

12/19/2025

V1.0

Table of Contents

1. Introduction.....	4
1.1 Purpose of This Document.....	5
1.2 References.....	5
2. Requirements.....	5
2.1 Functional Requirements.....	5
2.2 Non-Functional Requirements.....	6
3. Design.....	6
3.1 UML Diagram.....	6
4. Tech stack and Architecture.....	7
4.1. Google Apps Script (Backend).....	7
4.2 Google Workspace APIs.....	8
4.3 HTML/ CSS /Javascript (Frontend UI).....	8
5. User Interface.....	8
6. Status.....	9
6.1 Milestones.....	9
6.2. Looking Forward.....	9
Appendix A – Agreement Between Customer and Contractor.....	10
Appendix B – Team Review Sign-off.....	11
Appendix C – Document Contributions.....	12
Appendix D – System Requirements Specification.....	13
1. Introduction.....	13
1.1 Purpose of This Document.....	13
1.2. References.....	14
1.3. Purpose of the Product.....	14
1.4. Product Scope.....	14
Use Case Diagram.....	15
2. Functional Requirements.....	16
Create and Categorize.....	16
Search within Document by Tag.....	17
Create Custom Hashtag Template.....	18
Autocomplete Hashtag.....	19
Search/Filter by documents containing tags.....	20
SideBar Suggestions in Google Docs.....	21
Google Drive dashboard view.....	22
Export Tagged Notes.....	23
3. Non-Functional Requirements.....	24
4. User Interface.....	27

5. Deliverables.....	27
6. Open Issues.....	28
Appendix E – System Design Document.....	29
1 Introduction.....	29
1.1 Purpose of This Document.....	29
1.2 References.....	29
2 System Architecture.....	29
2.1 Architectural Design.....	30
2.2 Decomposition Description.....	31
3 Persistent Data Design.....	32
3.1 Database Descriptions.....	33
3.2 File Descriptions.....	34
4 Requirements Matrix.....	35
Appendix F – User Interface Design Document.....	35
1. Introduction.....	36
1.1 Purpose of This Document.....	36
1.2 References.....	36
2. User Interface Standards.....	36
2.1 Overall Screen Layout.....	37
2.1.1 Screen Areas Description.....	37
2.2 Common Components.....	38
2.2.1 Buttons.....	38
2.2.2 Input Fields.....	39
2.2.3 Icons.....	40
2.3 Color System.....	41
2.4 Typography.....	42
2.5 Spacing System.....	43
2.6 Navigation Patterns.....	43
2.6.1 Primary Navigation.....	43
2.6.2 Search and Filter.....	44
2.7 Error Handling.....	44
2.7.1 Inline Validation.....	44
2.7.2 Error Messages.....	45
2.7.3 System-Level Errors.....	45
2.8 Responsive Behavior.....	46
2.9 Loading States.....	46
2.10 Accessibility Standards.....	47
3. User Interface Walkthrough.....	48

3.1 Navigation Diagram.....	48
3.2 Screen Descriptions.....	49
3.2.1 [S1] Main View with Smart Sidebar.....	49
3.2.2 [S2] All Tags View.....	51
3.2.3 [S3] By Project View.....	52
3.2.4 [S3.1] Project Expanded View.....	54
3.2.5 [S3.2] Tag Details View.....	55
3.2.6 [S4] By Date View.....	58
3.2.7 [S5] Global Tags View.....	59
3.2.8 [M1] Edit Tag Modal.....	61
3.2.9 [M2] Delete Confirmation Modal.....	64
3.2.11 [S6] Command Palette.....	68
4. Data Validation.....	71
4.1 Data Validation Overview.....	71
4.2 Tag Management Data Items.....	71
4.3 Document Tagging Data Items.....	75
4.4 Metadata Data Items.....	76
4.5 Validation Rules by Input Type.....	77
4.5.1 Text Input Fields.....	77
4.5.3 Numeric Fields.....	77
4.6 Validation Timing.....	78
4.7 Error Message Standards.....	78
4.8 Data Sanitization.....	79
4.9 Data Constraints Summary.....	80

1. Introduction

The client requested a lightweight and intuitive to use plugin that will manage notes across different documents found in the user's Google Doc and Drive. They found other alternatives to be too invasive with their implementations, thus requesting to create a user friendly system that doesn't get in the way while making documents. The software will utilize a tag based system proposed by the client, that will allow users to effortlessly create tags that will easily keep track of text found underneath the tag. The only requirement for creating a tag is typing a unique character like @, #, \$, %, &, etc. then typing the name. (Ex. #budget, \$IEEE, %AI, Φtodo, &CMOS) Then any text found underneath will be associated with the tag until the user does a newline/pagebreak or type out a new/existing tag.

1.1 Purpose of This Document

The Critical Design Review Document provides an evaluation of the system's design before full-scale development begins. Its purpose is to confirm that the requirements are understood, the architecture is stable, and the proposed implementation approach is technically feasible. The CDR demonstrates that the team has completed the major design activities, validated the design choices against the project requirements, and produced a plan that can be executed within the available time and resources. By the end of the document, the client should have confidence that the system is well-defined and the project is ready to proceed into development and testing.

1.2 References

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail System Requirements Specification*

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail System Design Document*

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail User Interface Design Document*

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail Critical Design Review Document*

2. Requirements

Requirements for the product are split into 8 functional requirements and 14 non-functional requirements. The functional requirements are meant to let the software development team know what the software is expected to do, and when they have met the requirements specified by the client. The non-functional requirements will measure how well the software performs under various stress tests.

2.1 Functional Requirements

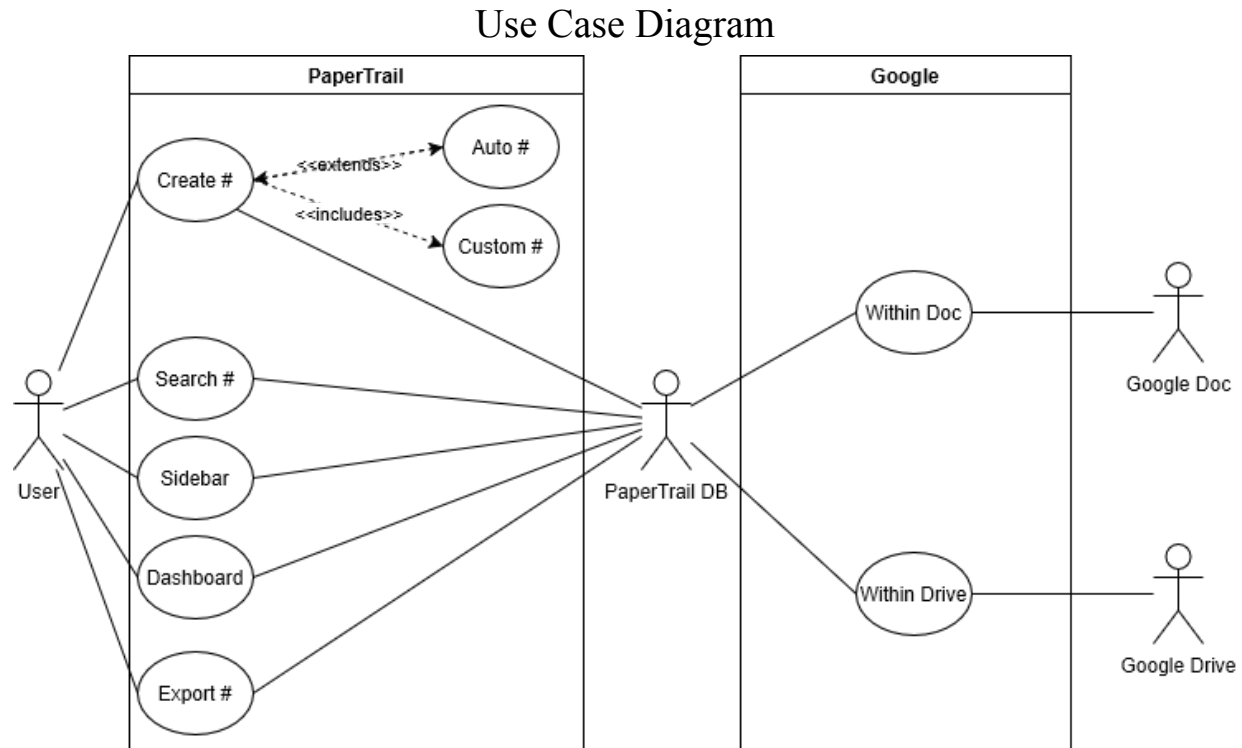
The 8 functional requirements are as follows: (FR-1) create tags of text inputs beginning with one of the recognized characters (#, @, \$) that will then be promptly categorized and saved, (FR-2) search for a given tag within a document, (FR-3) define and save new custom tags that did not previously exist, (FR-4) suggest tags after a recognized character is inputted and autofill tags after user selects a suggested tag, (FR-5) in a Google Drive scenario, filter between certain documents based on tags they contain, (FR-6) display a toggleable, non-intrusive side bar where existing tags in the document can be searched and different types of tags can be filtered through, (FR-7) in a Google Drive scenario, display a toggleable dashboard that lists all tags across documents, allows browsing by tag, and shows previews of related notes, (FR-8) export notes marked by a specified tag to a PDF or another Google Doc.

2.2 Non-Functional Requirements

The 14 non-functional requirements are as follows: (NFR-1) relating to FR-1, system shall tag and format lines within 2 seconds after user enters a recognized character (#, @, \$) in a document up to 50 pages, (NFR-2) relating to FR-1, all tagged lines shall display consistent font style, color, and metadata formatting across all users and sessions, (NFR-3) relating to FR-2, plugin shall display tag search results within 1 second for up to 500 tagged entries in a single document, (NFR-4) relating to FR-2, search results shall highlight matches clearly and maintain navigation state even after scrolling or editing, (NFR-5) relating to FR-3, the plugin shall allow a new tag to be created and saved with ≤ 3 user actions, (NFR-6) relating to FR-3, only authenticated users shall be able to create or modify shared tag templates; unauthorized users shall be denied access, (NFR-7) relating to FR-4, autocomplete suggestions shall appear within 0.5 seconds of typing a recognized character or partial text, ranking most-used tags first, (NFR-8) relating to FR-4, the autocomplete menu shall be intuitive and support convenience, allowing users to select suggestions using keyboard input (Tab/Enter), (NFR-9) relating to FR-5, the plugin shall interface reliably with Google Drive API and retrieve matching documents with $\geq 99\%$ success rate, (NFR-10) relating to FR-5, search queries and tag metadata shall never be transmitted outside of the user's authenticated Google environment, (NFR-11) relating to FR-6, the sidebar shall refresh contextual suggestions within 3 seconds after document edits exceeding 100 words, (NFR-12) relating to FR-6, sidebar shall occupy $\leq 25\%$ of screen width and be dismissible via toggle button at all times, (NFR-13) relating to FR-7, the plugin code shall use modular structures and be fully documented to support new feature updates in < 2 hours of dev time, (NFR-14) relating to FR-8, the plugin shall comply with Google Workspace publishing standards and FERPA data guidelines

3. Design

3.1 UML Diagram



PaperTrail is intended to be a lightweight and intuitive plugin for Google Doc and Drive, it will look for real time edits made to documents in order to find tags. These tags will consist of a name, date, and a string of characters in its metadata to allow the user to keep track of notes across spanning documents. The software will require permission from the user to allow access to their personal Google Docs and Drive in order to fulfill these functions. The software will mostly work in the background, while providing UI elements to allow the user more control over what the software deems as a tag, and noting where other tags exist across documents. If the user deems they don't want to see UI elements, they will have the option to collapse or disable the software how they see fit. The software is meant to be an assistance tool to keep track of notes for research projects.

4. Tech stack and Architecture

4.1. Google Apps Script (Backend)

Primary backend environment that powers the plugin.

Used for:

- Tag detection and parsing
- Document scanning and indexing
- Searching tags in a document
- Filtering documents in Drive
- Exporting tagged content
- Managing sidebar and dashboard data
- Storing lightweight settings
- Apps Script is required for Google Workspace add-ons and is the core of all backend logic.

4.2 Google Workspace APIs

These APIs enable reading, updating, and searching documents:

- Google Drive API
 - Search across documents for tag usage
 - Retrieve file metadata
 - Support the cross-document dashboard
- Google Docs API
 - Read and modify document content
 - Detect tag markers
 - Insert Metadata

4.3 HTML/ CSS /Javascript (Frontend UI)

Used through Apps Script HTML Service.

This builds the plugin's visual components:

- Sidebar UI

5. User Interface

The PaperTrail UI design is shown in detail in the UIDDD located in Appendix F. The sidebar is the main element of the user interface. It appears to the right of the google doc and includes:

1. Header
2. Search bar for tags
3. Tabs: All, Projects, Date, and Global
4. Footer for “help”

The UI also includes highlights for tag markers and metadata (date, author, etc.). There are also modal windows for editing existing tags, and confirmation of deletion.

6. Status

6.1 Milestones

By the end of the semester, the team has completed all major design-related milestones for PaperTrail. The System Requirements Specification, System Design Document, and User Interface Design Document have been fully drafted, reviewed, and finalized. In addition, the team has produced an initial working UI prototype for Google Docs that demonstrates the core layout and user interactions planned for the final system. These accomplishments confirm that the foundational design work is complete and that the project is positioned to enter full implementation at the start of next semester.

6.2. Looking Forward

PAPERTRAIL ROADMAP

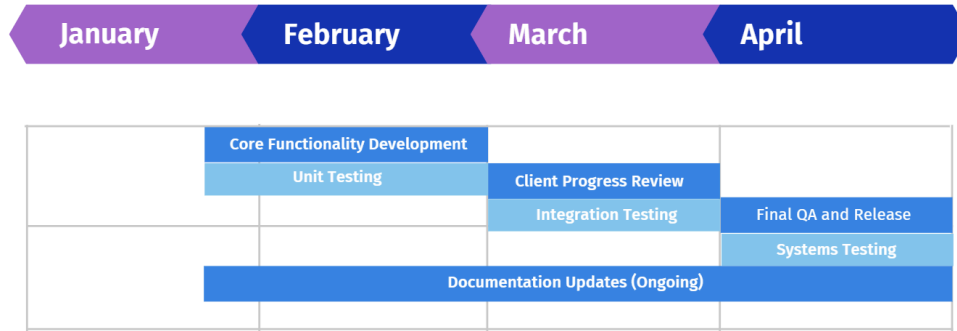


Figure 6.2.1 Proposed timeline for next semester

January to March: The team will implement the main operational features of the plugin, including tag searching, autocomplete behavior and tag based filtering for documents. This phase is instrumental as it lays the groundwork for the rest of the project. As we develop these features we will be continuously adding unit tests to verify and maintain working code as development continues upholding the working code at all times standard.

March to April: By march we plan to have completed all of our features and our goal will switch from development to refinement and bugtesting. We will take feedback from our client and make any necessary changes.

April to May: At this point the project should have reached the point where it can be released to end users. During this time the plugin will be deployed in its functional form for broader user access and we will collect real-world feedback, monitor performance and make any final adjustments before closing development.

Appendix A – Agreement Between Customer and Contractor

This document represents a formal agreement between the PaperTrail Development Team and Dr. Zihan Wu. The agreement confirms that both parties understand and accept the scope, objectives, and deliverables of the *PaperTrail Project*, which aims to create a document tracking and workflow automation system integrated with Google Docs and Google Drive. Both parties agree to collaborate throughout the project lifecycle, with the development team responsible for implementing the system according to the specifications outlined in this Critical Design Review Document, and the customer providing timely feedback and testing support.

In the event of future changes to this document, all revisions must be proposed in writing by either party. Any modification will require a review meeting between the customer and the contractor, and mutual written approval before updates are considered valid. Updated versions of the CDRD will be labeled clearly with version numbers and dates to ensure traceability.

Signatures

Name (Typed)	Signature	Date
Dr. Zihan Wu (Client)	<i>Zihan Wu</i>	12/18/25
Vasu Patel	<i>Vasu Patel</i>	12/16/25
Ben Yandell	<i>Benjamin Yandell</i>	12/16/25
Robert Kulow	<i>Robert Kulow</i>	12/15/25
Arius Ahmad	<i>Arius Ahmad</i>	12/16/25
Anthony Veilleux	<i>Anthony Veilleux</i>	12/16/25

Appendix B – Team Review Sign-off

All members of the PaperTrail Development Team have thoroughly reviewed this Critical Design Review Document and agree that it accurately represents the project's goals, scope, and requirements. Each team member confirms that the content and format of this document meet the expectations of both the team and the customer. While minor editorial or stylistic differences may exist, there are no major points of disagreement among the team regarding the material presented herein.

Team Member Signatures

Name (Typed)	Signature	Date
Vasu Patel	<i>Vasu Patel</i>	12/17/25
Ben Yandell	<i>Benjamin Yandell</i>	12/17/25
Robert Kulow	<i>Robert Kulow</i>	12/15/25
Arius Ahmad	<i>Arius Ahmad</i>	12/16/25
Anthony Veilleux	<i>Anthony Veilleux</i>	12/17/26

Appendix C – Document Contributions

The following table identifies each team member's contributions to the creation of this Critical Design Review Document. Each member has contributed to writing, editing, and reviewing the document. The percentages represent an estimate of each member's contribution to the total effort.

Team Member	Contributions	Percentage of Work
Vasu Patel	Assisted with Section 2 & 4	20%
Ben Yandell	Section 5, What has been done.	20%
Robert Kulow	Formatted document to keep consistency, worked on part 3 and implementation of other documents as appendices.	20%
Arius Ahmad	Completed section 2	20%
Anthony Veilleux	Completed Timeline Section, Tech stack and architecture, and purpose of the document	20%

Appendix D – System Requirements Specification

1. Introduction

This is a capstone project commissioned by Dr. Zihan Wu. This project will fulfill the capstone requirement for a Computer Science Bachelor's degree for Ben Yandell, Robert Kulow, Anthony Veilleux, Arius Ahmad, and Vasu Patel. The project is intended to give students a low-stakes opportunity to develop their emotional intelligence and software development skills. This project will aim to use JavaScript and Google Scripts to create a seamless and intuitive note assistance software natively for Google Docs and Google Drive that will help researchers organize their notes and write up documents.

1.1 Purpose of This Document

The purpose of this document is to outline the system requirements, so the development staff will be able to differentiate what they need to do, and when they are done. Specifically, this document will cover the functional and non-functional requirements of the software, what kind of user interface the software will utilize, how deliverables will be structured and presented, and any open issues that may present themselves during the development of the software. The software will be designed so that future users can easily utilize the software to assist their own research projects.

1.2. References

No references have been created so far, but this document will be updated with them as the project progresses.

1.3. Purpose of the Product

Previously, the client noticed that Google Docs doesn't natively support project management functions. When they tried to use third-party applications like Notion and Asana, they found them to be very convoluted and unintuitive to use. Prompting them to commission a native, intuitive plugin that works directly within Google Docs while writing. They proposed a hashtag system where any paragraph found below the hashtag is considered a note until a page break or

another hashtag. Thus, differentiating notes from actual documentation. Also, they requested a stretch goal: the ability to do quick commands with backslashes to bring up to-do tables and such on the fly.

1.4. Product Scope

This capstone project is aimed to be a lightweight expedition that gives the team members a glimpse of real-world software development processes. The scope being semi-detached as the project is split across two semesters, giving leeway for developmental obstacles to be quickly resolved in a low-stakes environment. The Ultimate goal is to have the finished product done at the end of the second semester.

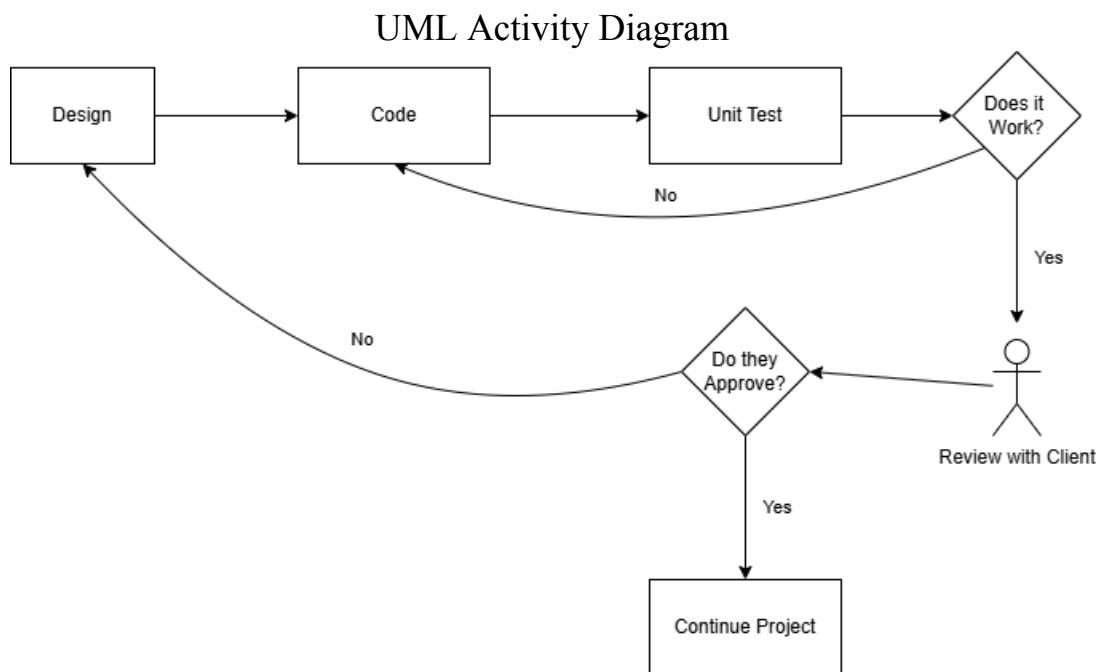


Figure 1: This diagram is a simple demonstration of our design flow we aim to use to gain user involvement from the client.

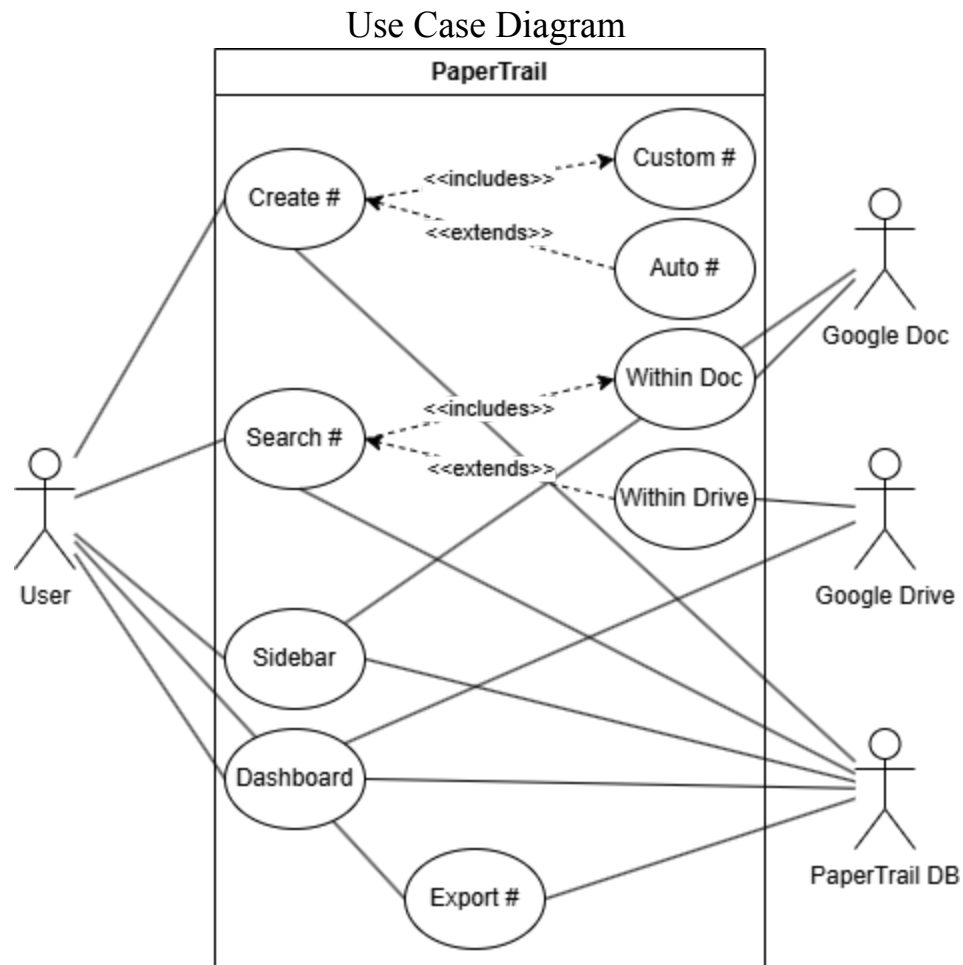


Figure 2: This diagram aims to highlight the interworking systems, and how they will react to user input from the functional requirements.

2. Functional Requirements

Each functional requirement will be in constant review as the software is designed and developed. They will let the software development team know what the software is expected to do, and when they have met the requirements specified by the client.

Number	01	
Name	Create and Categorize Nested Hashtags	
Summary	Format tag lines beginning with recognizing hashtags Ex. #Note, @Idea, \$Doc	
Priority	5	
Preconditions	Authorized user defined hashtags Document is editable	
Postconditions	The Tagged lines have consistent styling and readable tag metadata	
Primary Actor	Researcher/Dr. Zihan Wu	
Secondary Actors	Google Docs API	
Trigger	User types supported hashtag at line start or selects suggested tag from UI	
Main Scenario	Step	Action
	1	User selects #Idea
	2	Plugin detects tag on keypress
	3	Plugin stores “tag=Idea” in metadata
	4	Entry is then added to tag index
Extensions	Step	Branching Action:
	1a	Unknown tag: Plugin suggests closest known tag or includes customized tag option
	1b	User disabled auto-format: Only metadata added no visual style
Open Issues	Should there be case sensitivity?	
Tests	1. Typing “#Note” styles the line and adds “tag=Note” 2. Typing “@Time” styles the line and adds “tag=Time” 3. Typing “#Note @Time” styles the line and add “tag=Note, tag=Time” 4. Unknown tag triggers “suggested” 5. Tag index shows new entries made	

Number	02	
Name	Search within Document by Tag	
Summary	Navigate to each entry of selected tag via UI	
Priority	4	
Preconditions	Tag exists in a document Tag is indexed At least one tag exists.	
Postconditions	Cursor transfers to selected entry, current position is then updated	
Primary Actor	Researcher/Dr. Zihan Wu	
Secondary Actors	Google Docs API	
Trigger	User selects a tag or enters search criteria into the UI.	
Main Scenario	Step	
	1	User picks “#Citation” in UI
	2	Plugin lists all matches with a short preview
	3	User selects desired result and cursor moves to the entry of the doc
	4	UI highlights active item
Extensions	Step	
	1a	No results: Offer to create tag or show “help” on tagging
	1b	Large Document: Load the results present in the document at a slow rate so as to not overload processing, and allow the user to scroll to further entries in order to find their desired tag.
Open Issues	Results autoupdate as user types	
Tests	1. Clicking results jumps to the correct line 2. Search filter reduces list in real time as the criteria entered matches preexisting options. 3. Works with 500+ entries	

Number	03	
Name	Create Custom Hashtag Template	
Summary	Define, save and share custom tags, styles, and behaviors	
Priority	4	
Preconditions	User has permission to manage templates for their document	
Postconditions	New customized tags are available in autocomplete function and UI. New tags are shared to any collaborators	
Primary Actor	Researcher or Template owner	
Secondary Actors	Google Docs API	
Trigger	User opens “Manage Tags” and selects “New Tag”	
Main Scenario	Step	
	1	The user enters the tag name
	2	Plugin evaluates the uniqueness
	3	Template is saved to document or team
	4	Autocomplete refreshes to include the new custom tag
Extensions	Step	
	1a	Name conflict: Suggest alternative or allow "override" mechanic
	1b	Shared with collaborators: Tag is stored locally in the current document only, informing the user.
Open Issues	Who is able to create team-scope tags?	
Tests	1. New custom tag appears in autocomplete suggestion 2. Shared tag is visible to other collaborators upon refresh	

Number	04	
Name	Autocomplete Hashtag	
Summary	When user types a symbol '#' or partial identifier '@id' (for idea) plugin opens a suggestion list of known tags including default, custom, and shared. Selecting a suggestion inserts tag, applies style, and records metadata.	
Priority	5	
Preconditions	Plugin is installed Document is open and editable Tag index (default, custom, shared) is open and available for document	
Postconditions	Chosen tag is inserted at cursor position with consistent styling Tag metadata (tag=Idea, tag=Note) is stored and indexed for search and summary features.	
Primary Actor	Researcher or Template owner	
Secondary Actors	Google Docs API	
Trigger	User opens "Manage Tags" and selects "New Tag"	
Main Scenario	Step	
	1	User types '#' and starts entering wanted tag
	2	Plugin then detects and shows suggestions of what user may be referring to.
	3	The suggestion is prioritized by what the plugin believes is most relevant (ex. #No being #Note)
	4	User clicks desired autocomplete tag (or perhaps by using tab similar to other google autocompletion)
	5	Plugin inserts tag style and adds metadata
		The autocomplete suggestion list closes and focus goes back to current document
Extensions	Step	
	1a	If no matches: Present the "create new tag" option
	1b	User cancels: Close autocomplete suggestion.
Open Issues	1. Ranking the autocomplete suggestions a. Ex. frequency of used tag/most recent	
Tests	1. Halfway typing a tag '#No' shows '#Note' as top suggestion 2. Selecting a suggestion inserts its style and metadata 3. Pressing 'Esc' key closes autocomplete suggestion	

Number	05	
Name	Search/Filter by Documents containing Tags	
Summary	When a user types in the tag in Google Drive, it should retrieve the documents containing that tag.	
Priority	5	
Preconditions	Plugin is installed Documents with tags exist User is in Google Drive	
Postconditions	A list of Google documents that contain the queried tags are presented to the user.	
Primary Actor	Researcher	
Secondary Actors	Google Docs/Google Drive	
Trigger	User clicks on search bar within Google Drive	
Main Scenario	Step	
	1	When a user is in Google Drive, they click on the search bar
	2	The user then enters the tag(s) they want to search for
	3	The system then retrieves those documents and presents them to the user
Open Issues		
Tests	1. Search for Documents with tags that are known to exist 2. Search for Documents with tags that don't appear	

Number	06	
Name	Sidebar Suggestions in Google Docs	
Summary	Display a toggleable non intrusive sidebar that suggests related tags, recent /related notes or other auto detected elements like dates and names based on the current document content.	
Priority	3	
Preconditions	Plugin is installed Document is open and editable Document has data and tags for analysis	
Postconditions	Sidebar updates dynamically	
Primary Actor	Researcher or Student	
Secondary Actors	Google Docs API	
Trigger	User activates side-bar	
Main Scenario	Step	
	1	User activates side-bar
	2	Plugin displays tags with dates
	3	Sidebar lists suggestions and displays related documents if applicable
	4	User selects a suggestion
	5	Plugin inserts a suggestion
Extensions	Step	
	1a	Toggleable - User can toggle the sidebar on/off easily
	1b	User can disable autodetection
Open Issues		
Tests	1. Selecting a suggestion inserts tag correctly 2. Sidebar correctly displays related notes 3. Sidebar refreshes in real time	

Number	07	
Name	Google Drive Dashboard View	
Summary	Toggleable dashboard in Google Drive that lists all hashtags across documents, allows browsing by tag, and shows previews of related notes.	
Priority	4	
Preconditions	User is in Google Drive Plugin installed Data available	
Postconditions	Dashboard displays filtered results; clicking a note preview opens the document.	
Primary Actor	Researcher/Student	
Secondary Actors	Google Docs API	
Trigger	User accesses the dashboard via the plugin menu in Google Drive.	
Main Scenario	Step	
	1	User opens dashboard
	2	Plugin aggregates tags from accessible documents
	3	User filters by tag
	4	Dashboard shows notes previews
	5	User clicks on document preview and is sent to that document
Open Issues		
Tests	1. Filtering by tag shows only matching documents. 2. Preview accurately reflects note content. 3. Handles 100+ documents without lag.	

Number	08	
Name	Export Tagged Notes	
Summary	Allow users to export tagged notes (e.g., all #ToDo items) to PDF, or another Google Doc for sharing or backup.	
Priority	3	
Preconditions	User is in Google Drive Plugin installed Data available	
Postconditions	Export file generated and downloadable.	
Primary Actor	Researcher/Student	
Secondary Actors	Google Docs API	
Trigger	User selects the export option from the UI.	
Main Scenario	Step	
	1	User chooses tag and format
	2	Plugin compiles matching notes
	3	Generates and downloads files
Open Issues		
Tests	1. Export matches all tagged content 2. File opens correctly in target format 3. Handles 100+ notes	

3. Non-Functional Requirements

The non-functional requirements will measure how well the software performs under various stress tests. The tests will consist of recording the responsiveness and stability of the software over a period of time, as well as ensuring that no cybersecurity flaws are being violated. Throughout the development process, we aim to keep these requirements in mind during our design and testing phases.

NFR #	Related FR #	Priority (1–5)	Requirement Description	Verification / Test Method
NFR-01 Performance	FR-01: Create and Categorize	5	The system shall tag and format lines within 2 seconds after the user enters a hashtag (e.g., #Note or #Idea) in a document up to 50 pages.	Test-01: Time plugin response for 20 consecutive tag entries in a 50-page doc; verify formatting completes ≤ 2 seconds each.
NFR-02 Accuracy & Consistency	FR-01: Create and Categorize	4	All tagged lines shall display consistent font style, color, and metadata formatting across all users and sessions.	Test-02: Apply identical tags across 3 user accounts; confirm uniform style and metadata consistency.
NFR-03 Performance / Responsiveness	FR-02: Search within Document by Tag	5	The plugin shall display tag search results within 1 second for up to 500 tagged entries in a single document.	Test-03: Measure latency during 10 search queries in a test doc; verify ≤ 1 second response.
NFR-04 Usability	FR-02: Search within Document by Tag	4	Search results shall highlight matches clearly and maintain navigation state even after scrolling or editing.	Test-04: User testing with 5 participants; confirm navigation and highlight persist after edits.

NFR-05 Usability / Customization	FR-03: Create Custom Hashtag Template	3	The “Manage Tags” interface shall allow a new tag to be created and saved with ≤ 3 user actions.	Test-05: Observe test users creating a new tag; verify average ≤ 3 user actions.
NFR-06 Authentication	FR-03: Create Custom Hashtag Template	4	Only authenticated users shall be able to create or modify shared tag templates. Unauthorized users shall be denied access.	Test-06: Attempt tag creation without authentication; verify system denies request and logs event.
NFR-07 Predictive Performance	FR-04: Autocomplete Hashtag	5	Autocomplete suggestions shall appear within 0.5 seconds of typing “#” or partial text, ranking most-used tags first.	Test-07: Measure response time for 10 random hashtag queries; verify ≤ 0.5 second average delay.
NFR-08 Usability / Learnability	FR-04: Autocomplete Hashtag	4	The autocomplete menu shall be intuitive, allowing users to select suggestions using keyboard input (Tab/Enter).	Test-08: Observe 5 users performing tag insertions; confirm both selection methods function correctly.
NFR-09 Integration & Reliability	FR-05: Search/Filter by Documents Containing Tags	4	The plugin shall interface reliably with Google Drive API and retrieve matching documents with $\geq 99\%$ success rate.	Test-09: Conduct 100 Drive search operations; verify ≥ 99 results returned correctly.
NFR-10 Security / Privacy	FR-05: Search/Filter by Documents Containing Tags	5	Search queries and tag metadata shall never be transmitted outside of the user’s authenticated Google environment.	Test-10: Perform network packet inspection; verify no external API calls beyond Google endpoints.

NFR-11 (Performance)	FR-06: Sidebar Suggestions in Google Docs	3	The sidebar shall refresh contextual suggestions within 3 seconds after document edits exceeding 100 words.	Test-11: Edit 100+ words and measure sidebar update time $\leq$ 3 sec.
NFR-12 (Usability / Non-Intrusiveness)	FR-06: Sidebar Suggestions in Google Docs	4	Sidebar shall occupy $\leq$ 25% of screen width and be dismissible via toggle button at all times.	Test-12: Confirm layout constraint and visibility toggle in Chrome, Edge, Firefox.
NFR-13 (Maintainability)	FR-07: Collaborative Tag Sharing (if applicable)	3	The plugin code shall use modular structures and be fully documented to support new feature updates in < 2 hours of dev time.	Test-13: Developer inspection confirming code modules and documentation completeness.
NFR-14 (Compliance / External)	FR-08: Integration & Deployment	4	The plugin shall comply with Google Workspace publishing standards and FERPA data guidelines.	Test-14: Review package for compliance checklist and approval verification.

4. User Interface

See User Interface Design Document for PaperTrail to understand the user interface functionality and layout.

5. Deliverables

Upon completion, all deliverables will be presented to the client for approval in order to gain user involvement with the client. Deliverables will have hard and digital copies for file maintenance and to track project progress. Hard copies will be distributed to each team member and the client, while digital copies will be stored in the GitHub repository.

There will be hard copies of the following materials:

- Systems Requirement Specification (10/29/25)
- System Design Document (11/17/25)
- User Interface Design Document (12/03/25)
- Critical Design Review Document (12/19/25)
- Code Inspection Report (TBD)
- Administrator Manual (TBD)
- User Guide (TBD)
- Final Project Report (TBD)

There will be digital copies of the following materials:

- Systems Requirement Specification (10/29/25)
- System Design Document (11/17/25)
- User Interface Design Document (12/03/25)
- Critical Design Review Document (12/19/25)
- Code Inspection Report (TBD)
- Administrator Manual (TBD)
- User Guide (TBD)
- Final Project Report (TBD)
- All source code (TBD)
- The executable program (TBD)
- Any other software required for installation and execution of the delivered program. (TBD)

6. Open Issues

User Interface:

The user interface of the plugin is still to be finalized. After the first meeting with the client, they stated they wanted a minimal, nonintrusive UI. The plan moving forward is to work with the client using UI mockups and prototypes in hopes of gaining a better understanding of precisely what the client is looking for. This issue will continue to become a higher priority as we get closer to creating and finalizing the User Interface Design Document.

Appendix E – System Design Document

1 Introduction

This is a capstone project commissioned by Dr. Zihan Wu. This project will fulfill the capstone requirement for a Computer Science Bachelor's degree for Ben Yandell, Robert Kulow, Anthony Veilleux, Arius Ahmad, and Vasu Patel. As a researcher, the client has expressed frustration with the difficulty that comes with maintaining a single “main” document where progress, notes, and references are collected. They noticed that Google Docs doesn't natively support project management functions. When they tried third-party applications like Notion and Asana, they found them very convoluted and unintuitive. This project aims to offer an intuitive plugin that works directly within Google Docs while writing. The client proposed a tagging system in which any paragraph that follows a special character (“#”, “@”, etc.) is treated as a note until a page break or another tag. Thus, differentiating notes from actual documentation. The client also requested multiple types of tags (type of note, project it's under, date, etc.) to allow for more targeted searching functionalities. Also, they requested a stretch goal: the ability to do quick commands with backslashes while typing in the document to bring up to-do tables and such on the fly.

1.1 Purpose of This Document

The purpose of this document is to outline a high-level overview of the system architecture and the low-level design specifications. This document will clarify all technologies, tools, databases, etc. that are to be used for the development of the PaperTrail product and its requirements. This document will also go into detail about how said technologies will be structured and implemented, and what their purpose is.

1.2 References

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail System Requirements Specification*

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail System Design Document*

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail User Interface Design Document*

2 System Architecture

This section will demonstrate the functions and architecture of the plugin. How it will be utilized by the user, how it will react to the established systems provided by Google, and how it will avoid infringing on any cybersecurity policies. By the end of this section, readers should have a clear understanding of the scope and extent of this project.

2.1 Architectural Design

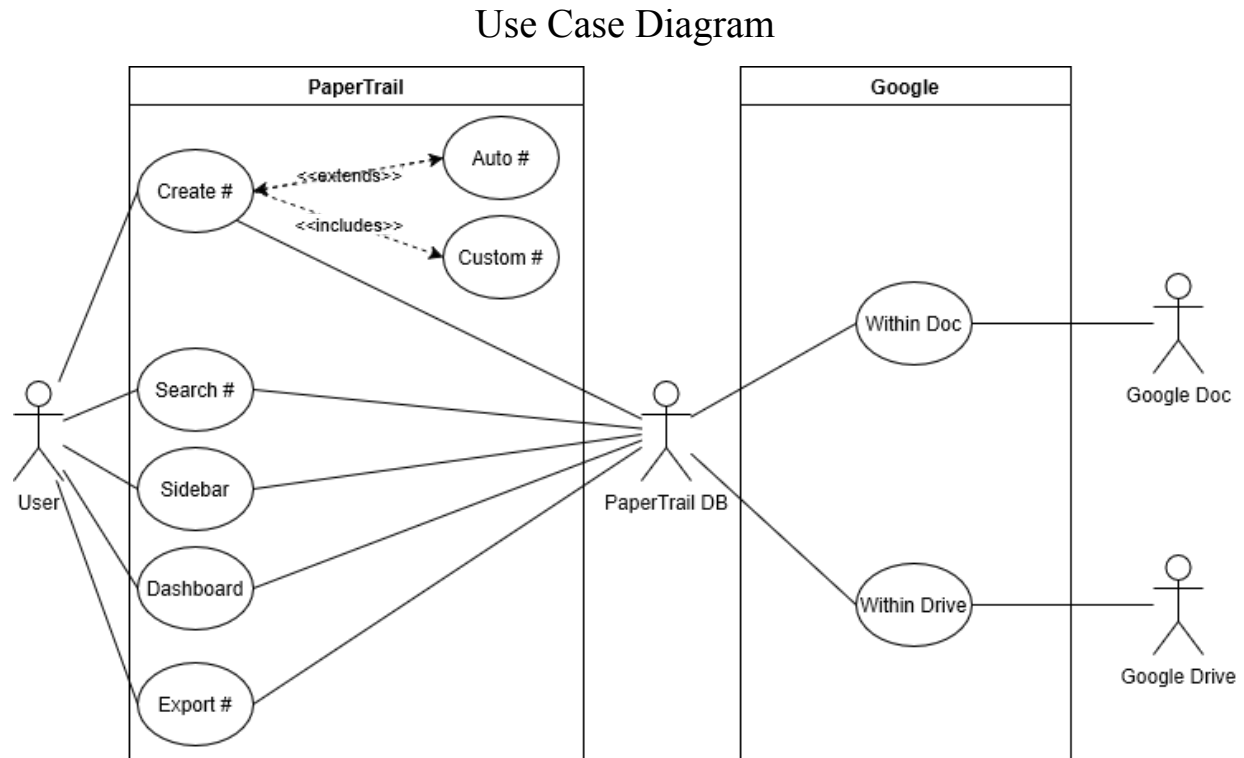


Figure 1: A use case diagram that shows the functions that the user will be able to access, and the systems the software will delegate for the user.

This use case diagram illustrates how the users will interact with the software, and how the software will react to Google's systems. The user will be able to interact with the front-end functions such as creating tags, searching tags, sidebar functionality, dashboard functionality, and a function to export tags. Each front-end component is linked to corresponding backend processes or systems. For example, the sidebar integrates with Google Docs, while the dashboard connects to Google Drive. Each function directly corresponds to the PaperTrail database.

The software will partially rely on Google's API for Google Docs and Drive. The plugin will host its own backend that is local to the machine. This will be done by using React to be a frontend system that will shape the way Google Docs looks by providing HTML and

CSS scripts. On the backend, additional functionality for Google Docs and Drive will be implemented via JavaScript and GoogleScript languages.

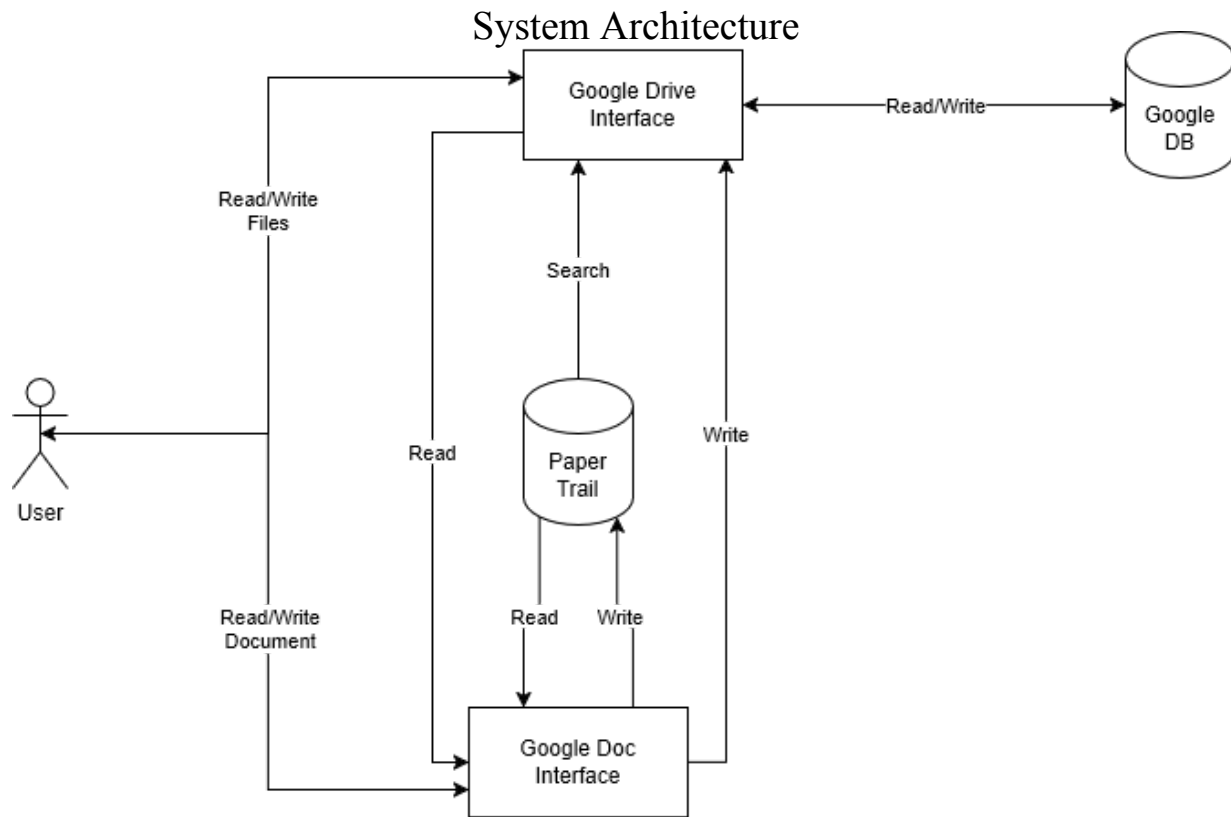


Figure 2: A system architecture diagram that shows how the user interacts with Google's systems will affect the software.

The system architecture diagram above illustrates that PaperTrail will have access to read and write privileges to the user's documents, and the search functionality provided by PaperTrail in the user's personal Google Drive. This will make it so that there are no infringements with Google's private database when Paper Trail makes updates to users' documents; The only instance where an update is pushed to Google's servers is through Google Drive, which Google Docs does automatically whenever changes are detected.

The paper trail system is meant to be an assistant software that helps the user write up documents, while easily having access to information across the documents they have in their Google Drive. Which is why PaperTrail doesn't have direct functionality with the user, instead working off the information provided by the user's documents found in their own Google Doc and Drive. This is done by letting PaperTrail read and make modifications that the user provides, and can search throughout their Google Drive to find tags that have been requested by the user in the document they are writing.

2.2 Decomposition Description

Class Diagram

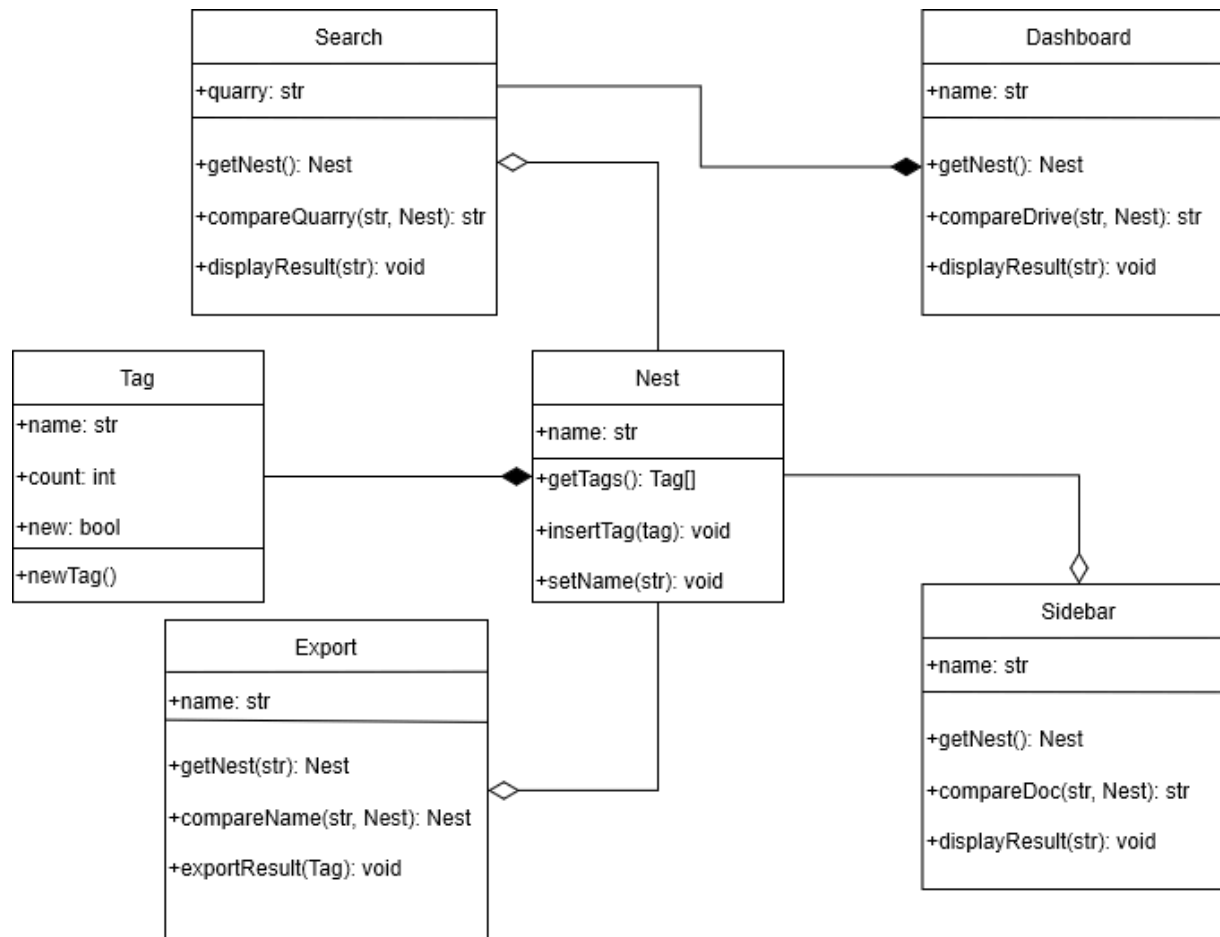


Figure 3: A class diagram that shows how each function will operate, and how they connect to the other functions when instantiated.

The Class Diagram above illustrates the classes that will be used by the software. Each Tag object will be categorized into a Nest object that will house an array of Tags that have been associated together by the user. Later will be accessed by the other Classes like Search, Export, Sidebar, and Dashboard. The Export class will take the requested nest of tags and export them onto a text file with their tagged notes, allowing the user to have them for personal use. The Search Class will take the queried input and compare it against the nest of tags objects, returning any that match the query, and displaying them to the user. Lastly, the Dashboard class found in the user's Google Drive, and the Sidebar class will be accessed while the user is writing in Google Docs.

3 Persistent Data Design

This section describes the permanent storage structure used by PaperTrail. The system uses Google's API to maintain records of documents, supported tags, and the locations where those tags appear within documents. This persistent data enables search functionality and autocomplete suggestions. The following section details each database table, its purpose, and the fields necessary for its implementation.

3.1 Database Descriptions

Database Diagram

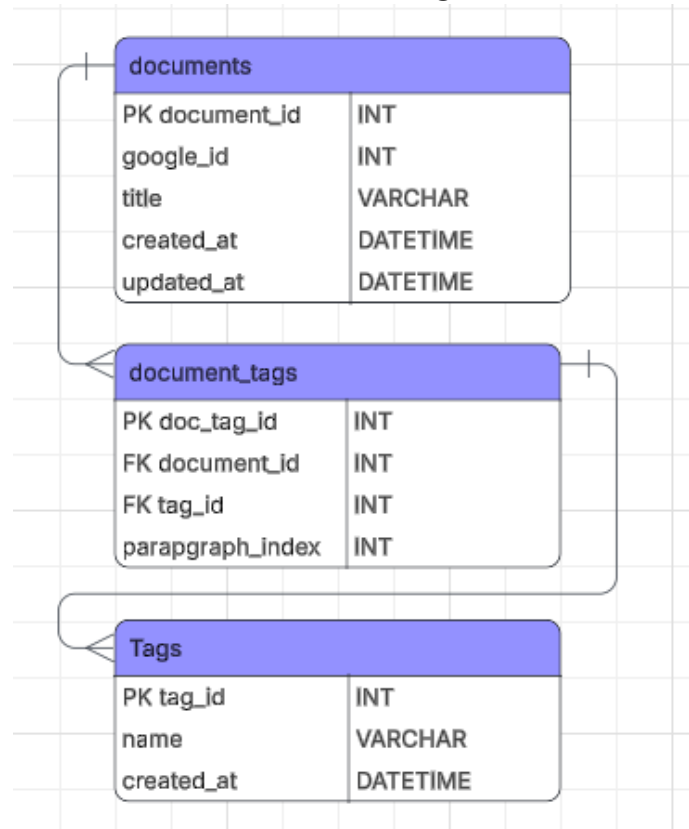


Figure 4: The database diagram shows how each table is related.

documents Table:

Stores basic information about every Google Doc that the plugin tracks.

Field Name	Purpose
document_id (Primary Key)	Internal unique ID for each document in the database.
google_id	The Google Docs unique identifier extracted from the document's URL. Used to reconnect to the document later.
title	The readable name of the document at the time it was first recorded.
created_at	Timestamp of when the system first indexed the document.
updated_at	The last time the plugin re-indexed the document or updated its metadata

document_tags Table:

This links specific tags to specific locations inside a document.

Field Name	Purpose
Doc_tag_id (Primary Key)	Unique ID for each tag assignment. One row = one tagged paragraph. Each tag has one string of characters.
Document_id (Foreign Key)	Which document the tag appears in.
Tag_id (Foreign Key)	Which tag was used
paragraph_index	The paragraph number/line number where the tag is located inside the Google Doc.

Tags Table:

Stores the tags to be used by the system.

Field Name	Purpose
Tag_id PK	Internal unique ID for each tag
name	The text of the tag (example: #Note).
created_at	Timestamp of when the tag was added to the system

3.2 File Descriptions

No flat file storage is used. All persistent data is stored directly in software, hosted locally on the user device. Google Docs content is accessed dynamically through the Google Docs API and is not saved locally.

4 Requirements Matrix

This section provides a matrix linking the functional requirements from the PaperTrail System Requirements Specification to their corresponding system components. Each requirement has a unique identifier that is mapped to the interfaces or modules that satisfy it.

The table below shows which architectural or design components (frontend modules, backend services, etc.) fulfill each requirement.

SRS ID	Name	Component/Module
1	Create and Categorize Nested Hashtags	Frontend - HTML & CSS Backend - Google Script, Tag Index System
2	Search within Document by Tag	Frontend (Search UI Component) Backend - (Search Handler), Google Docs API
3	Create Custom Hashtag Template	Template Manager (UI Modal), Tag Database
4	Autocomplete Hashtag	Frontend (Autocomplete Module) Backend Tag Priority System
5	Search/Filter by Documents Containing Tags	Backend (Google Drive API) Frontend Search Dashboard
6	Sidebar Suggestions in Google Docs	Frontend (Sidebar Component) Backend (Suggestion Engine)
7	Google Drive Dashboard View	Frontend - Dashboard UI within Drive Backend - Drive API
8	Export Tagged Notes	Backend (Export Handler), Drive API File Generator (PDF etc.)

Appendix F – User Interface Design Document

1. Introduction

This is a capstone project commissioned by Dr. Zihan Wu. This project will fulfill the capstone requirement for a Computer Science Bachelor's degree for Ben Yandell, Robert Kulow, Anthony Veilleux, Arius Ahmad, and Vasu Patel. As a researcher, the client has expressed frustration with the difficulty that comes with maintaining a single “main” document where progress, notes, and references are collected. They noticed that Google Docs doesn't natively support project management functions. When they tried third-party applications like Notion and Asana, they found them very convoluted and unintuitive. This project aims to offer an intuitive plugin that works directly within Google Docs while writing. The client proposed a tagging system in which any paragraph that follows a special character (“#”, “@”, etc.) is treated as a note until a page break or another tag. Thus, differentiating notes from actual documentation. The client also requested multiple types of tags (type of note, project it's under, date, etc.) to allow for more targeted searching functionalities. Also, they requested a stretch goal: the ability to do quick commands with backslashes while typing in the document to bring up to-do tables and such on the fly.

1.1 Purpose of This Document

The purpose of this User Interface Design Document is to define the visual and interactive components of the PaperTrail plugin. This document gives a detailed description of the user interaction components, including navigation structure, interactive elements, and visual styling. It translates the System Requirements Specifications functional requirements into interface design that will guide front-end implementation. This document ensures that the product is intuitive and aligned with the goals and user needs of the PaperTrail plugin.

1.2 References

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail System Requirements Specification*

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail System Design Document*

Arius, A., Anthony, V., Ben, Y., Robert, K., Vasu, P. (2025) *PaperTrail User Interface Design Document*

2. User Interface Standards

This section defines the design standards that will maintain consistency throughout the Research Tag Manager system. The interface follows Google Docs design principles to ensure familiarity for researchers and seamless integration with existing document workflows.

2.1 Overall Screen Layout

The Research Tag Manager system utilizes a three-panel layout as illustrated in Figure 1.

Software User Interface:

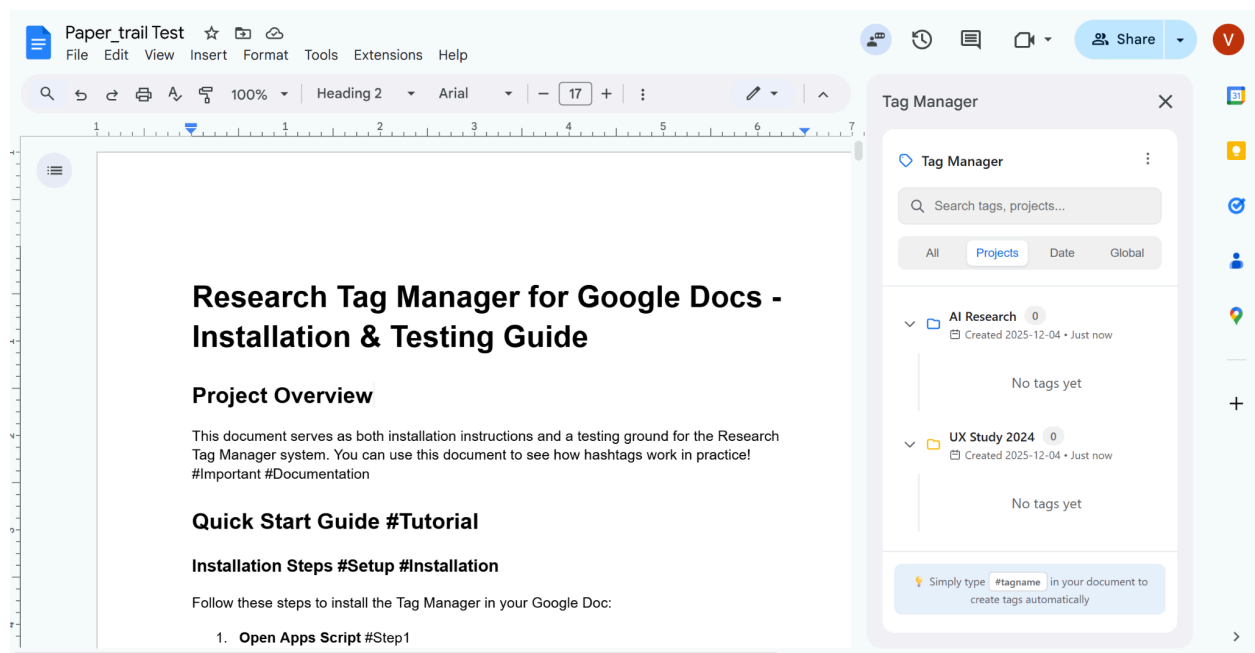


Figure 1. The system employs a three-panel layout with a fixed-height header (48px), flexible main content area, collapsible right sidebar (360px width), and optional footer (32px). The sidebar can be toggled to maximize document viewing space

2.1.1 Screen Areas Description

Top Navigation Bar:

- A thin bar at the top of the webpage that will stay at a fixed height on the screen, as seen in Figure 1. This nav bar will contain system branding (left), view mode switchers (center-left), and a keyboard shortcut button (right).

Smart Sidebar:

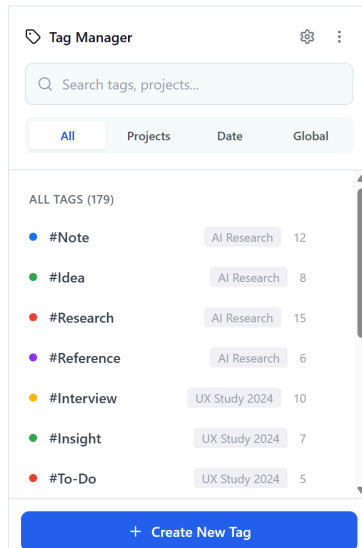


Figure 2. A sidebar that will appear to the right of a document being edited in real time. Allowing the user to organize tags found on the document, and others that have the relating tags as well

- This sidebar will appear on the right side of the screen when prompted open, moving the document over to the left slightly to make room for it. This bar also has the capability of being hidden (collapsible) to allow for the document content to take the focus. The sidebar will contain its UI elements in a stacked format from top to bottom. First, a title, a search bar, filter notches, which are listed from left to right where the user can filter displayed tags based on type, and, finally, a list of tags that fall under the filter or lack thereof, with scrollable capabilities if the list exceeds sidebar height. See Figure 3 for visual.

2.2 Common Components

2.2.1 Buttons

The system uses four button types as shown in Figure 4.

This design system defines four button styles:

Primary Buttons use a solid background with white text. They have no border, a 40px height, and an 8px corner radius. These are intended for high-priority actions such as creating tags or adding projects.

Secondary Buttons use a background with text. They include a 1px border in, a 36px height, and an 8px radius. These support lower-priority actions like canceling or navigating back.

Icon Buttons have a transparent background with icon color. They have no border, a 32px height, and a 4px radius. They are reserved for tool-related controls such as edit, delete, or settings.

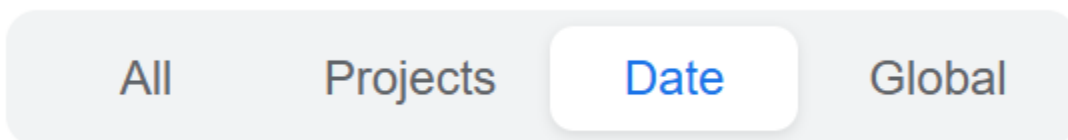


Figure 4. Button specifications define four distinct button types used throughout the system. All buttons shall have hover states with 10% darker background colors and focus states with 2px blue outline .

Button States:

- **Default:** As specified in table above
- **Hover:** Background darkens by 10%, cursor changes to pointer
- **Active/Pressed:** Background darkens by 20%, scale transforms to 98%
- **Disabled:** Opacity set to 40%, cursor changes to not-allowed
- **Focus:** 2px outline in, 2px offset

2.2.2 Input Fields

Element	Specification	Value
Height	Fixed	40px
Border	Default	1px solid
Border	Focus	2px solid
Border Radius	All corners	8px
Background	Default	
Background	Focus	
Padding	Horizontal	12px
Font Size	Body text	14px
Placeholder Color	Gray text	

Figure 5. Input field specifications ensure consistent form element styling. All input fields shall display icons 12px from the left edge when applicable (search icon, calendar icon).

Input Field States:

- **Empty:** Placeholder text visible in #9AA0A6
- **Active/Typing:** Border changes to #1A73E8, background to #FFFFFF
- **Filled:** Text in #202124, background remains #FFFFFF
- **Error:** Border changes to #EA4335, helper text appears below in red
- **Disabled:** Opacity 40%, background #F1F3F4, cursor not-allowed

2.2.3 Icons

The system uses Lucide React icon library with the following standards:

Context	Size	Color	Stroke Width
Navigation Icons	20px	#5F6368	2px
Tag Indicators	16px	Tag color	2px
Metadata Icons	12px	#5F6368	2px
Action Buttons	16px	Inherit	2px
Status Indicators	12px	Status color	2px

Figure 6. Icon sizing and coloring standards maintain visual hierarchy and ensure accessibility. Icons shall always be accompanied by tooltips on hover after 500ms delay.

2.3 Color System

Color Name	Hex Value	Usage
Primary Blue	#1A73E8	Primary actions, active states, links
Success Green	#34A853	Success messages, positive indicators
Warning Yellow	#FBBC04	Warning messages, caution states
Error Red	#EA4335	Error messages, delete actions
Accent Purple	#9334E9	Accent elements, special tags
Info Cyan	#0891B2	Informational messages, highlights
Gray 900	#202124	Primary text, headings
Gray 600	#5F6368	Secondary text, icons
Gray 300	#DADCE0	Borders, dividers
Gray 100	#F1F3F4	Subtle backgrounds
Gray 50	#F8F9FA	Section backgrounds
White	#FFFFFF	Main background, cards

Figure 7. The color palette follows Google's Material Design principles to ensure accessibility and visual consistency. All color combinations shall meet WCAG 2.1 AA contrast requirements (minimum 4.5:1 for normal text).

2.4 Typography

Style Name	Font Size	Font Weight	Line Height	Letter Spacing	Usage
Heading 1	32px	700 (Bold)	40px (1.25)	0	Page titles
Heading 2	24px	600 (Semibold)	32px (1.33)	0	Section headers
Heading 3	18px	600 (Semibold)	24px (1.33)	0	Subsection headers
Body Large	16px	500 (Medium)	24px (1.5)	0	Emphasized body text
Body	14px	400 (Regular)	20px (1.43)	0	Standard body text
Body Small	13px	400 (Regular)	18px (1.38)	0	Secondary information
Caption	12px	600 (Semibold)	16px (1.33)	0.5px	Labels, metadata
Overline	11px	600 (Semibold)	16px (1.45)	1px	Category labels (uppercase)

Figure 8. Typography scale ensures consistent text hierarchy throughout the system. The system shall use system fonts: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Google Sans', 'Roboto', sans-serif.

2.5 Spacing System

The system uses an 8px base unit for all spacing calculations.

Token Name	Value	Common Usage
space-4	4px	Tight gaps between related icons
space-8	8px	Gaps between form elements
space-12	12px	Standard element padding
space-16	16px	Section padding, card spacing
space-24	24px	Large section gaps
space-32	32px	Major section divisions
space-48	48px	Page padding
space-64	64px	Extra large spacing

Figure 9. All spacing in the system shall use multiples of the 8px base unit to maintain consistent rhythm and alignment.

2.6 Navigation Patterns

2.6.1 Primary Navigation

The system uses the following navigation patterns:

Sidebar Navigation:

- User navigates through tag views using four filter tabs: All, Projects, Date, Global
- Clicking a tab changes the content area below while maintaining scroll position
- Active tab displays with white background and blue text (#1A73E8)
- Tab transitions use 200ms ease-in-out animation

Hierarchical Navigation:

- Projects expand/collapse using chevron icons (ChevronRight/ChevronDown)
- Tags within projects expand to show metadata and tagged items
- Maximum nesting depth: 3 levels (Project > Tag > Items)
- Expansion state persists during session

Keyboard Navigation:

- Tab key moves focus through interactive elements in reading order
- Arrow keys navigate within lists
- Enter/Space activates focused buttons or toggles expansion
- Escape closes modals or command palette
- ⌘+K (Ctrl+K on Windows) opens command palette from any screen

2.6.2 Search and Filter

Search Behavior:

- Search field appears at top of sidebar
- Search activates on keystroke (no submit button required)
- Results filter in real-time with 150ms debounce
- Search queries match against tag names, project names, and item titles
- Clear button (X icon) appears when search has content
- Empty search shows all results

Filter Behavior:

- Filters are mutually exclusive (only one view mode active)
- Clicking a filter tab immediately updates content area
- Filter state persists across page refreshes using localStorage
- Active filter indicated by white background and blue text

2.7 Error Handling

2.7.1 Inline Validation

State	Border Color	Helper Text Color	Icon	Behavior
Valid	#34A853	#34A853	Checkmark	Shows after successful input
Invalid	#EA4335	#EA4335	Alert Circle	Shows on blur if invalid
Warning	#FBBC04	#FBBC04	Alert Triangle	Shows for non-critical issues

Figure 10. Validation states provide immediate feedback to users. Error messages shall appear below input fields with 4px top margin and shall describe the specific problem and required correction.

Validation Timing:

- Validation occurs on blur (when user leaves input field)
- For passwords and critical fields, validation also occurs on submit
- Real-time validation activates after first blur event
- Success indicators appear 300ms after valid input

2.7.2 Error Messages

Error messages shall follow this structure:

Format: [Problem] + [Required Action]

Examples:

- "Tag name is required. Please enter a tag name."
- "Tag name already exists. Please choose a different name."
- "Project name must be 3-50 characters. Please adjust the length."

Error Message Display:

- Position: Directly below the related input field
- Font: 12px regular, color #EA4335
- Icon: Alert circle icon 12px, aligned left
- Animation: Fade in over 150ms
- Duration: Remains visible until error corrected

2.7.3 System-Level Errors

Toast Notifications:

- Appear in top-right corner, 16px from edges
- Width: 360px fixed
- Auto-dismiss after 5000ms (errors), 3000ms (success)
- User can manually dismiss using X button
- Maximum 3 toasts stacked vertically

Error Types:

- **Connection Error:** "Unable to connect to server. Please check your internet connection."
- **Save Error:** "Failed to save changes. Please try again."
- **Delete Error:** "Cannot delete tag. Tag is currently in use."
- **Permission Error:** "You do not have permission to perform this action."

2.8 Responsive Behavior

Breakpoint	Width	Sidebar Behavior	Layout Changes
Desktop Large	$\geq 1440\text{px}$	Fixed visible 360px	Full three-panel layout
Desktop	1024-1439px	Fixed visible 320px	Sidebar width reduces
Tablet	768-1023px	Overlay drawer	Sidebar overlays on toggle
Mobile	$< 768\text{px}$	Full-screen modal	Sidebar takes full viewport

Figure 11. The system adapts to different viewport sizes while maintaining core functionality. Sidebar behavior changes at defined breakpoints to optimize screen real estate.

Responsive Rules:

- Minimum supported width: 360px
- At tablet size and below, sidebar toggles via hamburger menu
- Font sizes shall not scale below defined minimums
- Touch targets shall be minimum 44x44px on mobile devices
- Hover states replaced with active states on touch devices

2.9 Loading States

Component Loading:

- Skeleton screens display during initial data load
- Skeleton matches layout of actual content
- Loading duration $> 500\text{ms}$ triggers skeleton display
- Spinner icon (20px) used for button loading states

Page Loading:

- Full-page loader for navigation between major sections
- Progress bar at top of viewport (3px height, #1A73E8)
- Indeterminate animation for unknown duration

- Minimum display time: 500ms to prevent flashing

2.10 Accessibility Standards

The system shall comply with WCAG 2.1 Level AA standards:

Keyboard Accessibility:

- All interactive elements accessible via keyboard
- Focus indicators visible with 2px outline
- Logical tab order following reading flow
- Skip links provided for main content areas

Screen Reader Support:

- All images include alt text
- Form inputs associated with labels
- ARIA labels on icon-only buttons
- Status messages announced with aria-live regions

Color Contrast:

- Text contrast minimum 4.5:1 for normal text
- Large text (18px+) minimum 3:1 contrast
- Interactive elements minimum 3:1 contrast
- Error states distinguishable without color alone

3. User Interface Walkthrough

This section provides a comprehensive walkthrough of the Research Tag Manager interface, including navigation flows and detailed screen descriptions. The system centers around the Smart Sidebar interface with multiple view modes for tag organization.

3.1 Navigation Diagram

Figure 12 illustrates the primary navigation paths through the Research Tag Manager system.

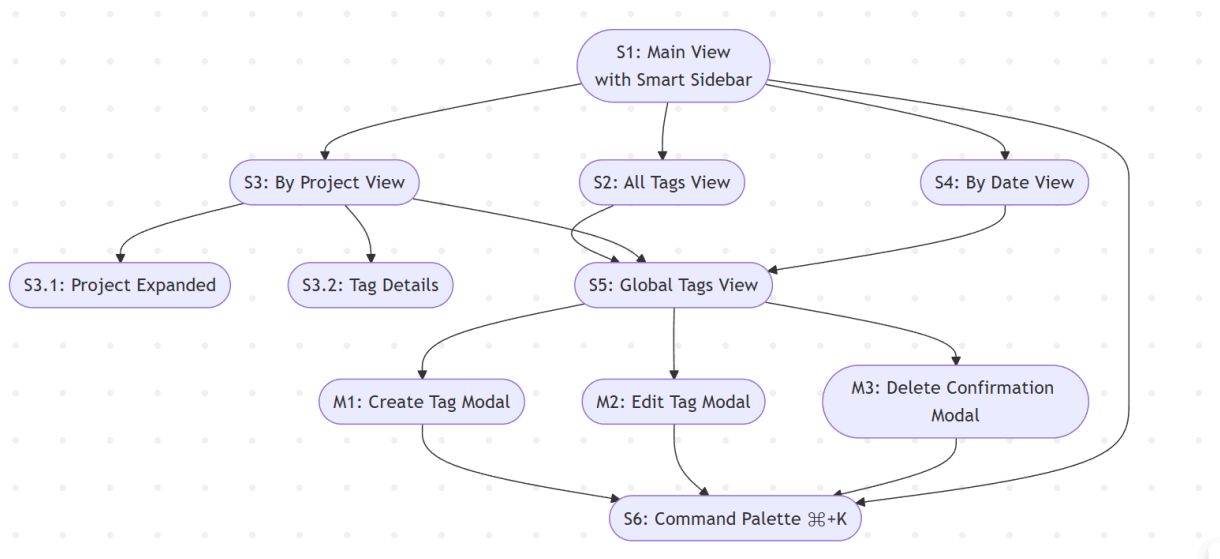


Figure 12. The navigation diagram shows the primary user flows through the Research Tag Manager. Users start at the Main View (S1) and can navigate to four different view modes via filter tabs. Each view provides access to modals for tag management actions (M1-M3). The Command Palette (S6) is accessible from any screen using the keyboard shortcut ⌘+K (Ctrl+K on Windows).

Navigation Flow Description:

The system employs a single-page application architecture with state-based view switching. Users begin at the Main View (S1), which displays the document area and Smart Sidebar. From S1, users navigate to different organizational views using the filter tab bar:

- **All Tags View (S2):** Displays a flat list of all tags across projects
- **By Project View (S3):** Shows hierarchical project-based organization
- **By Date View (S4):** Organizes tags chronologically
- **Global Tags View (S5):** Shows system-wide priority and status tags

Within the By Project View (S3), users can expand projects (S3.1) to reveal project-specific tags, which can be further expanded to show detailed metadata (S3.2). All views provide access to modal dialogs for creating (M1), editing (M2), and deleting (M3) tags. The Command Palette (S6) can be invoked from any screen using the keyboard shortcut.

3.2 Screen Descriptions

3.2.1 [S1] Main View with Smart Sidebar

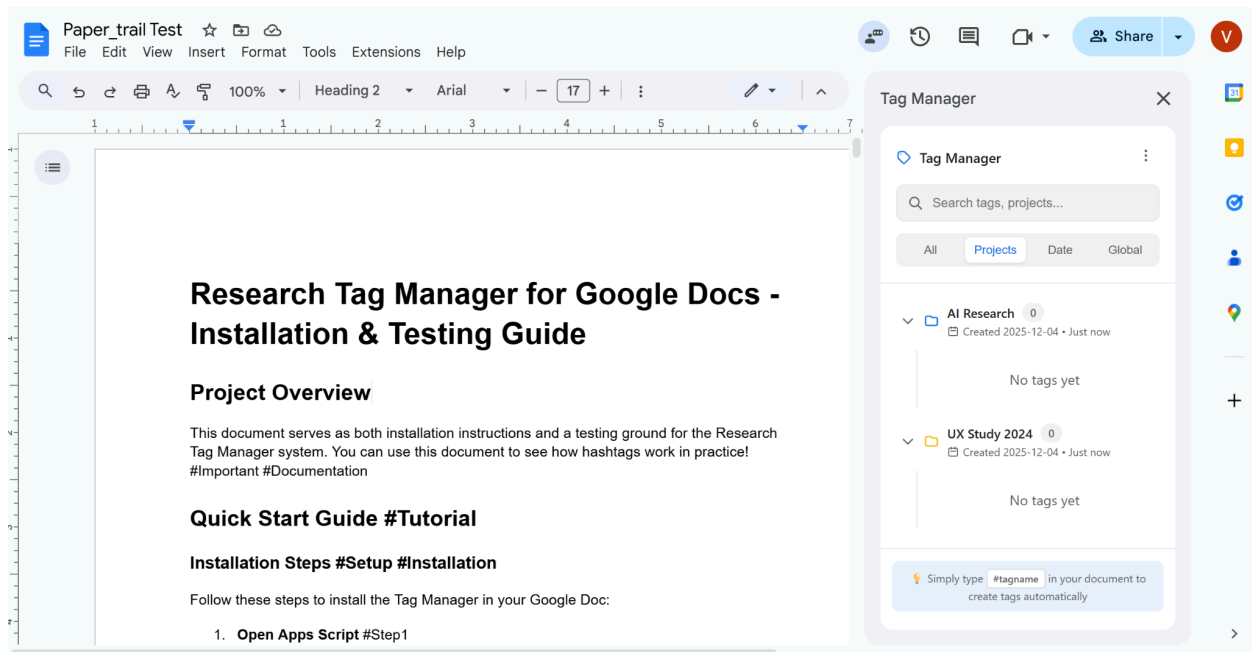


Figure 13. The Main View presents the primary working interface with the document editing area occupying the left portion and the Smart Sidebar fixed to the right edge. The sidebar provides tag management functionality without interrupting document workflow.

Screen Elements:

A. Top Navigation Bar:

- **Location:** Fixed at top of viewport, full width
- **Height:** 48px
- **Elements:**

- System branding (left): Tag icon + "Research Tag Manager" text
- "Enhanced" badge in blue (#1A73E8)
- Keyboard shortcut button (right): Shows "⌘ K" with gray background

B. Document Area:

- **Location:** Left side, flexible width
- **Contents:**
 - Project indicator: Folder icon + "AI Research" in blue
 - Document title: "Neural Network Architecture Study" (32px, bold)
 - Metadata row: Last edited date, author name, inline tag chips
 - Document content: Rich text with inline tag references
 - Tagged sections: Highlighted blocks with left border accent

C. Smart Sidebar:

- **Location:** Right edge, 360px width
- **Sections (top to bottom):**
 - Header: "Tag Manager" title with Settings and More icons
 - Search bar: Search icon + "Search tags, projects..." placeholder
 - Filter tabs: Four tabs (All, Projects, Date, Global)
 - Content area: Displays current view based on active filter
 - Footer: Two action buttons (Create New Tag, Add Project) + keyboard hint

Navigation Actions:

- Click filter tabs to switch between view modes (navigates to S2-S5)
- Click "⌘ K" button or press keyboard shortcut to open Command Palette (S6)
- Click Settings icon to access system preferences
- Click tag items in sidebar to filter document content

User Interaction Flow:

1. User views document content in main area
2. User searches for tags using search bar
3. User switches between view modes using filter tabs
4. User clicks specific tags to highlight related content in document
5. User creates new tags using "Create New Tag" button (opens M1)

3.2.2 [S2] All Tags View

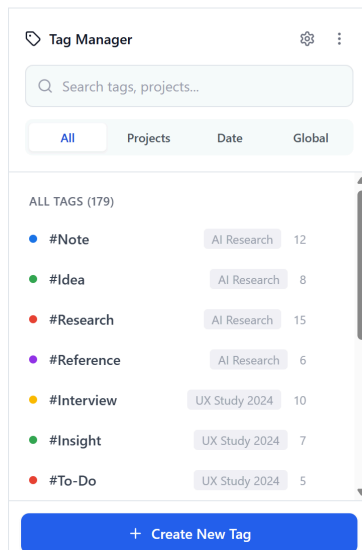


Figure 14. The All Tags View presents a comprehensive flat list of every tag in the system, regardless of project or date. Each tag displays its associated project as a badge and usage count.

Screen Elements:

Sidebar Content Area:

- **Header:** "All Tags (65)" - shows total tag count
- **List Structure:** Vertical scrollable list
- **Each Tag Item Contains:**
 - Color indicator dot (8px, left-aligned)
 - Tag name with # prefix (14px, medium weight)
 - Project badge (gray background, small text)
 - Usage count (gray text, right-aligned)
 - Chevron right icon (appears on hover)

Example Tags Displayed:

- #Note | AI Research | 12

- #Idea | AI Research | 8
- #Interview | UX Study 2024 | 10
- #Feature | Product Development | 9
- [Divider line]
- #Important | (global) | 25
- #Urgent | (global) | 12

Interaction Behavior:

- Hover on tag item: Background changes to #F8F9FA, chevron appears
- Click tag item: Filters document to show only items with that tag
- Search field filters list in real-time as user types
- Scroll: Smooth scrolling with visible scrollbar on hover

Navigation:

- Click "Projects" tab to navigate to S3
- Click "Date" tab to navigate to S4
- Click "Global" tab to navigate to S5
- Click any tag to filter document content

3.2.3 [S3] By Project View

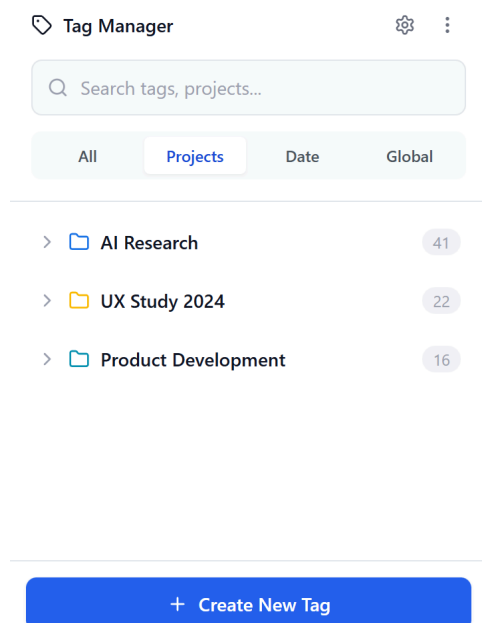


Figure 15. The By Project View organizes tags hierarchically under their parent projects. Users can expand and collapse projects to focus on specific areas of work.

Sidebar Content Area:

- **List Structure:** Hierarchical tree with collapsible nodes

Project Items (Collapsed State):

- Chevron right icon (4px, indicates expandable)
- Folder icon (16px, colored by project)
- Project name (14px, semibold)
- Tag count badge (gray background, shows total tags in project)
- Metadata row (12px, gray text):
 - Calendar icon + "Created [date]"
 - Bullet separator
 - Last activity timestamp

Example Projects:

1. **AI Research** (Blue folder)
 - Created 2024-01-15 • 2 hours ago
 - Badge: 41 tags
2. **UX Study 2024** (Yellow folder)
 - Created 2024-02-01 • 30 mins ago
 - Badge: 22 tags
3. **Product Development** (Cyan folder)
 - Created 2024-01-25 • 4 hours ago
 - Badge: 16 tags

Interaction Behavior:

- Click chevron or project name: Expands project to show tags (navigates to S3.1)
- Hover project: Background changes to #F8F9FA
- Projects remember expansion state during session

Navigation:

- Click collapsed project to expand (navigates to S3.1)

- Click "All" tab to return to S2
- Click "Date" tab to navigate to S4
- Click "Global" tab to navigate to S5

3.2.4 [S3.1] Project Expanded View

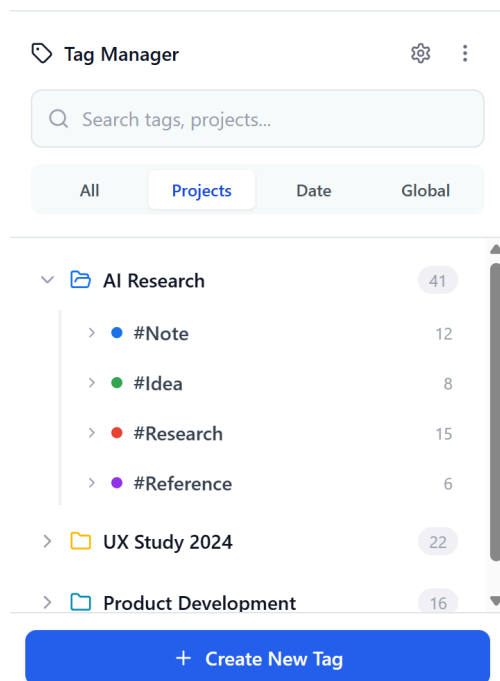


Figure 16. When a project is expanded, the view reveals all tags belonging to that project with a vertical line indicating hierarchy. Each tag can be further expanded to show detailed metadata and tagged items.

Project Header (Same as S3):

- Now shows chevron down icon (indicating expanded state)
- Folder open icon (instead of closed folder)
- Project name, count badge, and metadata row unchanged

Tag List (Indented with left border):

- **Visual Hierarchy:**
 - 24px left margin from project name
 - 2px gray border (#E8EAED) on left side
 - 8px left padding from border

Individual Tag Item:

- Chevron right icon (expandable indicator)
- Color dot (8px, matches tag color)
- Tag name with # prefix (14px, medium)
- Usage count (12px, gray, right-aligned)
- Edit and Delete icons (appear on hover)

Interaction Behavior:

- Click project header: Collapses project back to S3 state
- Click tag chevron: Expands tag to show metadata (navigates to S3.2)
- Hover tag item: Edit and Delete icons fade in
- Click Edit icon: Opens Edit Tag Modal (M2)
- Click Delete icon: Opens Delete Confirmation (M3)

Navigation:

- Click tag to expand metadata (navigates to S3.2)
- Click project header to collapse
- Click different project to expand it and collapse current

3.2.5 [S3.2] Tag Details View

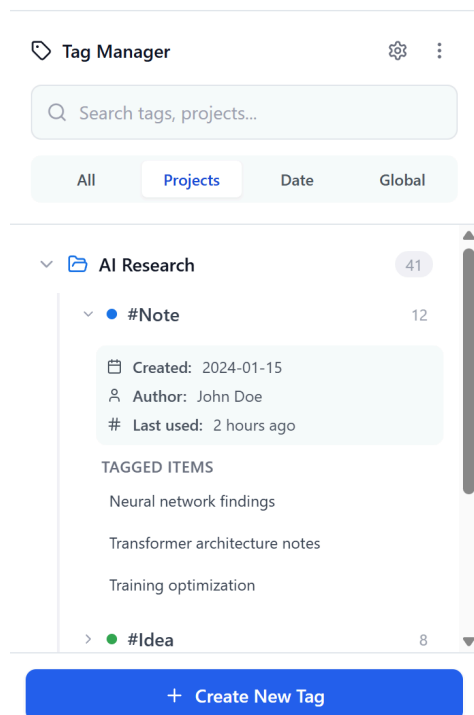


Figure 17. The Tag Details View displays comprehensive metadata about the selected tag including creation information, author, and a list of all items currently tagged with it. This view provides full context for tag usage.

Tag Header:

- Chevron down icon (indicates expanded)
- Color dot indicator
- Tag name with # prefix
- Usage count
- Edit and Delete action icons (visible on hover)

Metadata Panel:

- **Background:** Light gray (#F8F9FA)
- **Border radius:** 8px
- **Padding:** 8px
- **Contents (three rows):**
 - Row 1 - Created Date:**
 - Calendar icon (12px, gray)
 - "Created:" label (bold, 12px)
 - Date value (12px, gray)
 - "at" connector
 - Time value (12px, blue #1A73E8, bold)
 - **Row 2 - Author:**
 - User icon (12px, gray)
 - "Author:" label (bold, 12px)
 - Author name (12px, gray)
 - **Row 3 - Last Used:**

- Hash icon (12px, gray)
- "Last used:" label (bold, 12px)
- Relative time (12px, gray)

Tagged Items Section:

- **Header:** "TAGGED ITEMS" (11px, semibold, uppercase, gray)
- **List:** Vertical list of document titles
- **Each Item:**
 - 2px left border (transparent, becomes blue on hover)
 - Title text (12px, gray)
 - Hover state: Background #E8F0FE, border #1A73E8

Interaction Behavior:

- Click tag header: Collapses tag back to S3.1 view
- Click Edit icon: Opens Edit Tag Modal (M2)
- Click Delete icon: Opens Delete Confirmation (M3)
- Click tagged item: Navigates to that item in document
- Hover tagged item: Background highlights, left border appears

Navigation:

- Click tag header to collapse
- Click tagged item to jump to document location
- Click different tag to expand it and collapse current

3.2.6 [S4] By Date View

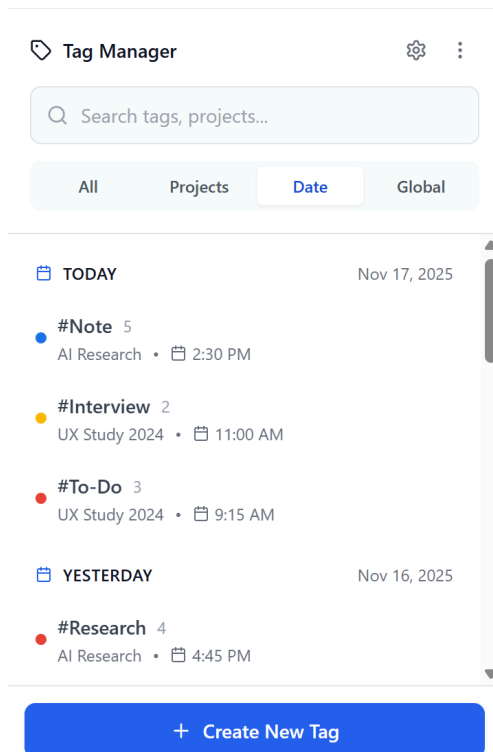


Figure 18. The By Date View organizes tags chronologically into intuitive time periods, allowing users to track recent activity and find tags based on when they were created or last used.

Each section consists of:

- **Sticky Header:**
 - Calendar icon (12px, blue)
 - Period label (11px, semibold, uppercase)
 - Date range (11px, gray, right-aligned)
 - Background: White (stays visible during scroll)
 - Z-index: 10

Time Period Organization:

1. **TODAY** - Nov 17, 2025
2. **YESTERDAY** - Nov 16, 2025
3. **THIS WEEK** - Nov 11-16, 2025
4. **LAST WEEK** - Nov 4-10, 2025
5. **EARLIER THIS MONTH** - Nov 1-3, 2025
6. **OLDER** - Before Nov 2025

Tag Items within Each Period:

- Color dot indicator (8px)
- Tag name and count (first row)
- Project name + timestamp (second row, 11px, gray)
 - Format: "[Project] •  [Time]"
- Chevron right icon (hover only)

Interaction Behavior:

- Scroll: Section headers remain sticky at top
- Hover tag item: Background changes to #F8F9FA, chevron appears
- Click tag: Filters document to show only items with that tag
- Click chevron: Shows quick preview of tag details

Navigation:

- Click "All" tab to navigate to S2
- Click "Projects" tab to navigate to S3
- Click "Global" tab to navigate to S5
- Scroll to view different time periods

3.2.7 [S5] Global Tags View

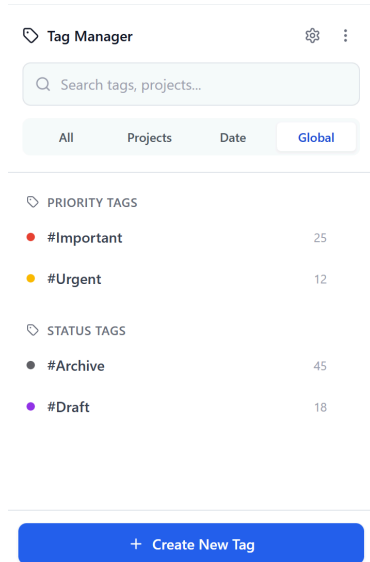


Figure 19. The Global Tags View displays system-wide tags that apply across all projects. Tags are categorized into Priority tags (Important, Urgent) and Status tags (Archive, Draft).

Priority Tags Section:

- **Header:**
 - Tag icon (12px)
 - "PRIORITY TAGS" label (11px, semibold, uppercase)
 - Top margin: 8px

Tag List:

- Each tag displays:
 - Color dot (8px)
 - Tag name with # prefix (14px, medium)
 - Usage count (12px, gray)
 - Chevron right icon (hover only)

Status Tags Section:

- **Header:** Same format as Priority Tags
- **Label:** "STATUS TAGS"
- **Top margin:** 24px from previous section

Tag Details:

- **Important:** Red dot (#EA4335), for high-priority items
- **Urgent:** Yellow dot (#FBB03B), for time-sensitive items
- **Archive:** Gray dot (#5F6368), for completed/archived items
- **Draft:** Purple dot (#9334E9), for work-in-progress items

Interaction Behavior:

- Hover tag: Background changes to #F8F9FA
- Click tag: Filters document to show items with that global tag
- Tags can be combined with project-specific tags
- Click chevron: Shows tag usage across all projects

Navigation:

- Click "All" tab to navigate to S2
- Click "Projects" tab to navigate to S3
- Click "Date" tab to navigate to S4
- Click any tag to filter document content

3.2.8 [M1] Edit Tag Modal

Tag Name *

Research 8/50

Only letters, numbers, hyphens, and underscores

Tag Color *

Blue Green Yellow Red (selected) Purple Teal

Assign to Project

AI Research

Description

For research-related documents and findings 43/200

Optional

Tag Usage Information

This tag is currently used in **15 items**

Last modified: Nov 15, 2025 at 3:30 PM

Created by: John Doe

Figure 20. The Edit Tag Modal allows users to modify existing tag properties including name, color, project assignment, and description. All fields are pre-populated with current tag values.

Modal Structure:

- **Overlay:** Full viewport, rgba(0,0,0,0.2) background with 8px blur
- **Modal Container:**
 - Width: 480px
 - Background: White
 - Border radius: 16px
 - Box shadow: 0 20px 60px rgba(0,0,0,0.3)
 - Position: Centered in viewport

Modal Header:

- **Title:** "Edit Tag: #[TagName]" (24px, semibold)
- Example: "Edit Tag: #Research"
- **Close Button:** X icon (top right)

Modal Body:

Field 1 - Tag Name (Required):

- Label: "Tag Name" with red asterisk
- Input field specifications:
 - Placeholder: "Enter tag name (e.g., Research, Note)"
 - Height: 40px
 - Max length indicator: "0/50 characters"
- Prefix indicator: # symbol in gray (automatically prepended)

Field 2 - Color Selection (Required):

- Label: "Tag Color" with red asterisk
- Color palette grid: 6 colors in 2 rows
- Each color swatch:
 - Size: 48px × 48px
 - Border radius: 8px
 - Selected state: 3px border in #1A73E8
 - Hover state: Scale to 105%

- Colors available:
 - Blue (#1A73E8), Green (#34A853), Yellow (#FBBC04)
 - Red (#EA4335), Purple (#9334E9), Cyan (#0891B2)

Field 3 - Project Assignment (Optional):

- Label: "Assign to Project"
- Dropdown select:
 - Placeholder: "Select a project (optional)"
 - Options: List of existing projects + "None (Global Tag)"
 - Height: 40px
 - Chevron down icon on right

Field 4 - Description (Optional):

- Label: "Description"
- Textarea:
 - Placeholder: "Add a description for this tag (optional)"
 - Height: 80px
 - Max length: 200 characters
 - Character counter: "0/200"
 - Resize: Vertical only

Additional Information Panel:

- **Background:** Light blue (#E8F0FE)
- **Padding:** 12px
- **Border radius:** 8px
- **Content:**
 - "This tag is currently used in 15 items"
 - "Last modified: Nov 15, 2025 at 3:30 PM"
 - "Created by: John Doe"

Modal Footer:

- **Buttons (right-aligned):**
 - Cancel button: "Cancel" text
 - Delete button: Secondary style with red text, "Delete Tag"
 - Save button: Primary style, "Save Changes" text

Interaction Behavior:

- Click overlay: Closes modal (with unsaved changes warning if applicable)

- Click X button: Closes modal
- Press Escape key: Closes modal
- Tab through fields: Focus moves in logical order
- Required fields: Show error state if submitted empty
- Create button: Disabled (40% opacity) until Tag Name and Color selected
- Character counters: Update in real-time as user types
- Success: Modal closes, new tag appears in sidebar, success toast appears
- Delete button: Opens Delete Confirmation modal (M2)
- Save button: Validates changes before saving
- If tag name changed: Checks for duplicate names
- Success: Modal closes, tag updates in sidebar, success toast appears
- Cancel: Closes without saving, changes discarded

Validation Rules:

- Tag name cannot be empty
- Tag name must be 1-50 characters
- Tag name cannot contain special characters except hyphen and underscore
- Tag name must be unique within selected project (or globally if no project)
- Color selection is mandatory
- Description limited to 200 characters
- Additional check: If tag name changed, must not conflict with existing tags
- Cannot delete required fields that were previously filled

Navigation:

- Click Save: Validates and updates tag, closes modal, returns to previous view
- Click Delete: Opens Delete Confirmation modal (M2)
- Click Cancel: Closes modal, discards changes, returns to previous view

3.2.9 [M2] Delete Confirmation Modal

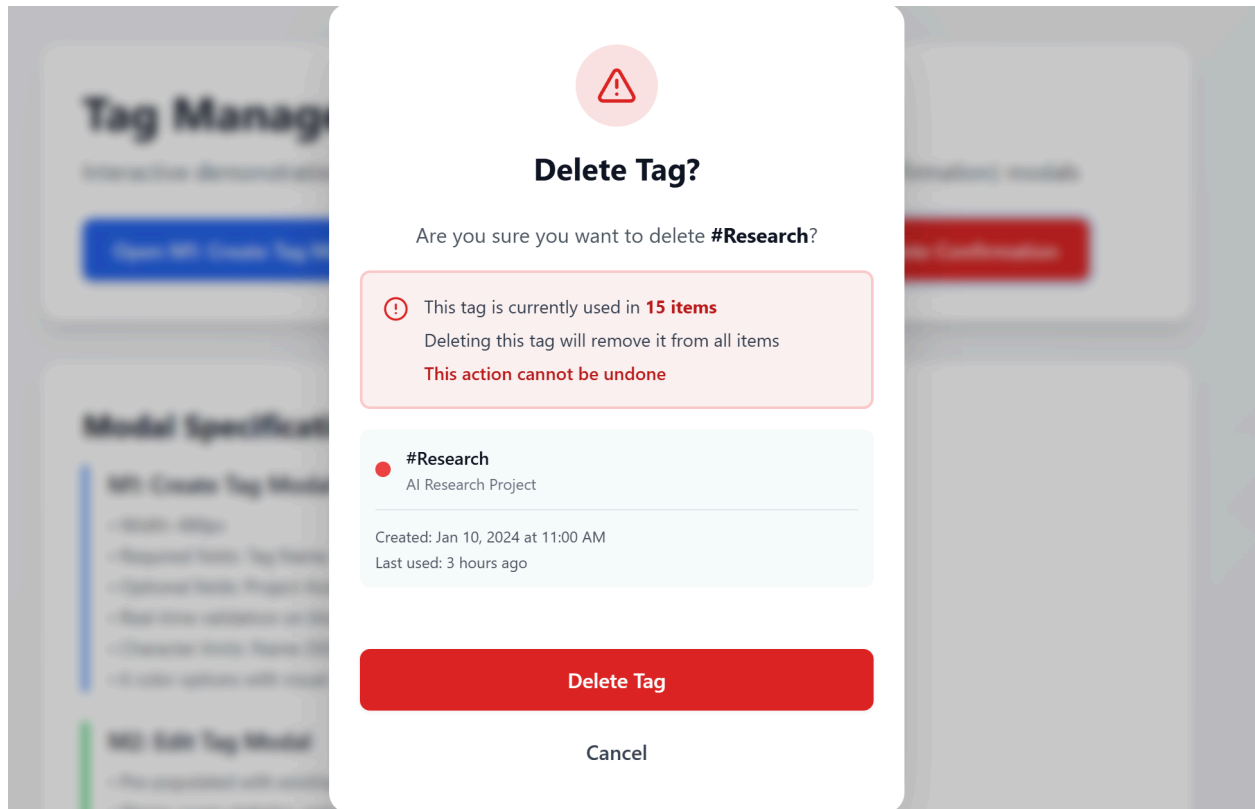


Figure 21. The Delete Confirmation Modal presents a warning dialog before permanently deleting a tag. It shows the impact of deletion and requires explicit confirmation.

Modal Structure:

- **Overlay:** Same as M1 and M2
- **Modal Container:**
 - Width: 400px (narrower than other modals)
 - Background: White
 - Border radius: 16px
 - Centered in viewport

Modal Header:

- **Icon:** Alert triangle in red circle (48px)
- **Title:** "Delete Tag?" (24px, semibold)
- **Padding:** 24px 24px 0 24px
- **Alignment:** Centered

Modal Body (Padding: 24px):

Warning Message:

- Text: "Are you sure you want to delete #[TagName]?"
- Font: 16px, regular, gray (#5F6368)
- Alignment: Centered

Impact Information Panel:

- **Background:** Light red (#FDECEA)
- **Border:** 1px solid #EA4335
- **Border radius:** 8px
- **Padding:** 16px
- **Icon:** Alert circle icon (red)
- **Content:**
 - "This tag is currently used in [X] items"
 - "Deleting this tag will remove it from all items"
 - "This action cannot be undone"
- **Font:** 14px, regular
- **Line spacing:** 1.5

Tag Details:

- **Background:** #F8F9FA
- **Padding:** 12px
- **Border radius:** 8px
- **Content:**
 - Tag name with color dot
 - Project name (if applicable)
 - Creation date
 - Last used date

Modal Footer:

- **Padding:** 0 24px 24px 24px
- **Buttons (full width, stacked):**
 - Delete button:
 - Background: #EA4335 (red)
 - Text: "Delete Tag"
 - Width: 100%

- Height: 44px
 - Font: 16px, semibold
- Cancel button:
 - Background: Transparent
 - Text: "Cancel"
 - Width: 100%
 - Height: 44px
 - Font: 16px, medium
 - Top margin: 8px

Interaction Behavior:

- Click Delete:
 - Shows loading spinner on button
 - Deletes tag from system
 - Removes tag from all tagged items
 - Closes modal
 - Shows success toast: "Tag deleted successfully"
 - Returns to previous view with tag removed
- Click Cancel: Closes modal, returns to Edit Tag modal (M2) if came from there, otherwise returns to sidebar
- Click overlay: Same as Cancel
- Press Escape: Same as Cancel
- Focus: Delete button receives focus on modal open (requires deliberate action)

Validation Rules:

- No validation required (confirmation action)
- Delete operation is permanent

- System updates all references immediately

Navigation:

- Click Delete: Executes deletion, closes all modals, returns to main view (S1)
- Click Cancel: Closes this modal, returns to Edit Tag modal (M2) if applicable

3.2.11 [S6] Command Palette

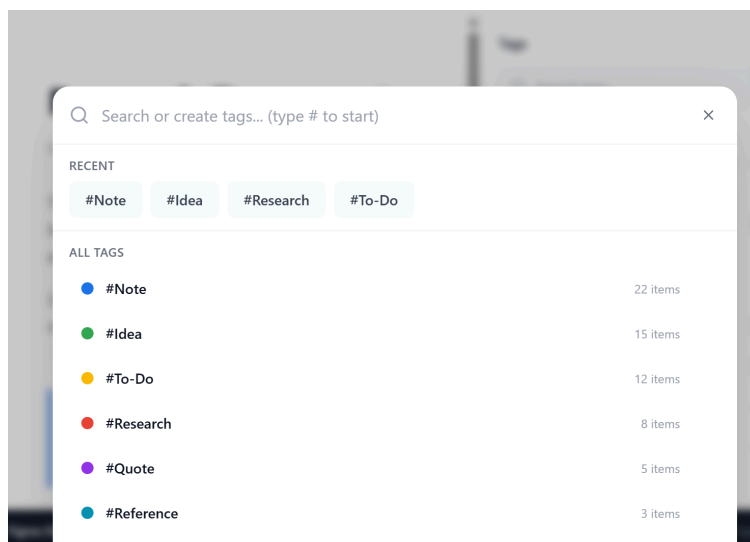


Figure 23. The Command Palette provides keyboard-driven quick access to all tag management functions. It can be invoked from any screen using $\text{⌘}+\text{K}$ (Ctrl+K on Windows) and allows users to search, create, and navigate tags without using the mouse.

Overlay:

- **Background:** rgba(0,0,0,0.2) with 8px backdrop blur
- **Coverage:** Full viewport
- **Z-index:** 2000 (above all other elements)

Palette Container:

- **Width:** 640px
- **Position:** Top-center, 128px from top of viewport
- **Background:** White
- **Border radius:** 16px
- **Box shadow:** 0 20px 60px rgba(0,0,0,0.3)

Search Header:

- **Padding:** 16px
- **Border bottom:** 1px solid #E8EAED

Search Input:

- **Icon:** Search icon (20px, gray, left-aligned)
- **Placeholder:** "Search or create tags... (type # to start)"
- **Font:** 16px, regular
- **Height:** 48px
- **Border:** None
- **Focus:** Auto-focused on open
- **Close button:** X icon (right-aligned, 16px)

Recent Section:

- **Padding:** 12px 16px
- **Border bottom:** 1px solid #E8EAED
- **Header:** "RECENT" (11px, semibold, uppercase, gray)
- **Tag Pills:** Horizontal scrollable row
 - Each pill:
 - Padding: 8px 16px
 - Background: #F8F9FA
 - Border radius: 8px

- Hover: Background #E8EAED
- Font: 14px, medium

Suggestions Section:

- **Padding:** 12px 16px
- **Max height:** 384px
- **Overflow:** Vertical scroll
- **Header:** "ALL TAGS" (11px, semibold, uppercase, gray)

Tag List Items:

- Each item contains:
 - Color dot (12px, left-aligned)
 - Tag name with # prefix (14px, medium)
 - Usage count: "[X] items" (12px, gray)
 - Chevron right icon (16px, right-aligned, visible on hover)
- **Hover state:** Background #F8F9FA
- **Selected state:** Background #E8F0FE
- **Padding:** 12px
- **Border radius:** 8px

Footer:

- **Padding:** 12px 16px
- **Background:** #F8F9FA
- **Border top:** 1px solid #E8EAED

Keyboard Shortcuts Display:

- **Left side:**
 - "↑↓ Navigate" with arrow key icons
 - "↵ Select" with enter key icon
- **Right side:**
 - "Esc Close" with escape key icon
- **Font:** 12px, gray
- **Key indicators:** White background, 1px border, 4px radius

Interaction Behavior:

- **Open:** Press ⌘+K (Ctrl+K) from any screen
- **Type:** Real-time filtering of tag list (150ms debounce)
- **Arrow keys:** Navigate through suggestions (highlights items)

- **Enter key:** Selects highlighted tag or creates new if typed with #
- **Escape key:** Closes palette, returns to previous screen
- **Click outside:** Closes palette
- **Recent tags:** Shows last 4 used tags for quick access
- **Empty search:** Shows all tags
- **Search with #:** Indicates intention to create new tag
 - Footer changes to show "Press Enter to create #[typed_name]"

Search Behavior:

- Matches against tag names, project names
- Case-insensitive search
- Partial matches highlighted in results
- Results sort by relevance, then by usage count
- If search matches no tags: Shows "No tags found. Press Enter to create #[query]"

Navigation:

- Select tag: Closes palette, filters document to selected tag, returns to S1
- Create new tag: Closes palette, opens Create Tag modal (M1) with name pre-filled
- Press Escape: Closes palette, returns to previous screen state
- Click outside: Same as Escape

4. Data Validation

This section specifies all data items that users can enter into the Research Tag Manager system. Each data item is defined with its data type, constraints, format requirements, and validation rules.

4.1 Data Validation Overview

All user input in the Research Tag Manager system undergoes validation at two stages:

1. **Client-side validation:** Occurs in real-time or on blur events to provide immediate feedback
2. **Server-side validation:** Occurs before data persistence to ensure data integrity

Validation errors shall be displayed inline below the relevant input field with descriptive error messages that specify the problem and required correction.

4.2 Tag Management Data Items

Table 1: Tag Creation and Editing Data Fields

Field ID	TG-01	TG-02	TG-03	TG-04
Field Label	Tag Name	Tag Color	Project Assignment	Tag Description
Data Type	String	String (Hex)	String (ID) or NULL	String
Required	Yes	Yes	No	No
Min Length	1	7	0	0
Max Length	50	7	36	200
Format	Alphanumeric, hyphen, underscore only	#RRGGBB	UUID v4 format	Free text
Validation Rules	Must be unique within project scope; Cannot contain spaces; Cannot start with numbers; Case-insensitive comparison	Must be one of six predefined colors: #1A73E8, #34A853, #FBBC04, #EA4335, #9334E9, #0891B2	Must reference existing project ID or be NULL for global tags	No special restrictions; Allows all Unicode characters
Error Messages	"Tag name is required." / "Tag name must be 1-50 characters." / "Tag name can only contain	"Please select a color for this tag." / "Invalid color selection."	"Selected project does not exist." / "Invalid project ID format."	"Description must be 200 characters or less."

	letters, numbers, hyphens, and underscores." / "A tag with this name already exists in this project."			
--	---	--	--	--

Table 2: Project Management Data Fields

Field ID	PJ-01	PJ-02	PJ-03
Field Label	Project Name	Project Color	Project Description
Data Type	String	String (Hex)	String
Required	Yes	Yes	No
Min Length	3	7	0
Max Length	50	7	500
Format	Alphanumeric, spaces, hyphens allowed	#RRGGBB	Free text

Validation Rules	Must be unique system-wide; Cannot start or end with spaces; Cannot contain special characters except hyphens and spaces	Must be one of six predefined colors	No special restrictions
Error Messages	"Project name is required." / "Project name must be 3-50 characters." / "A project with this name already exists." / "Project name cannot start or end with spaces."	"Please select a color for this project."	"Description must be 500 characters or less."

Table 3: Search and Filter Data Fields

Field ID	SF-01	SF-02
Field Label	Search Query	Filter Selection
Data Type	String	Enum
Required	No	Yes
Min Length	0	N/A
Max Length	100	N/A
Format	Free text	One of: "all", "projects", "date", "global"

Validation Rules	Accepts all characters; Trims leading/trailing whitespace	Must be one of four predefined values
Error Messages	N/A (Search has no validation errors)	N/A (Radio button selection, cannot be invalid)

4.3 Document Tagging Data Items

Table 4: Inline Tag Application Data Fields

Field ID	DT-01	DT-02	DT-03	DT-04
Field Label	Tag Selection	Tagged Content Start	Tagged Content End	Tagged Content End
Data Type	String (ID)	Integer	Integer	String
Required	Yes	Yes	Yes	Yes
Min Length	36	0	0	1

Max Length	36	2147483647	2147483647	10000
Format	UUID v4	Non-negative integer	Non-negative integer	Free text
Validation Rules	Must reference existing tag ID	Must be valid character position in document; Must be less than end position	Must be valid character position in document; Must be greater than start position	Cannot be empty; Maximum 10,000 characters per tagged selection
Error Messages	"The selected tag does not exist." / "Tag has been deleted."	"Invalid content selection." / "Start position must be before end position."	"Invalid content selection." / "The end position must be after the start position."	"Cannot tag empty selection." / "Tagged selection exceeds a maximum length of 10,000 characters."

4.4 Metadata Data Items

Table 5: System-Generated Metadata Fields

Field ID	Field Label	Data Type	Format	Generated By	Validation Rules
MD-01	Created Date	DateTime	ISO 8601 (YYYY-MM-DDT HH:mm)	System on creation	Cannot be modified by user; Must be valid timestamp
MD-02	Created Time	Time	HH:mm AM/PM	System on creation	Cannot be modified by user; Derived from Created Date

MD-03	Last Modified Date	DateTime	ISO 8601 (YYYY-MM-DDT HH:mm)	System on update	Cannot be manually set; Must be equal to or after Created Date
MD-04	Last Used Date	DateTime	ISO 8601 (YYYY-MM-DDT HH:mm)	System on tag application	Cannot be manually set; Updates when tag applied to content
MD-05	Author	String	Full name or email	System from logged-in user	Cannot be manually changed; Set to current user on creation
MD-06	Usage Count	Integer	Non-negative integer	System calculation	Cannot be manually set; Calculated from number of tagged items
MD-07	Last Activity	String	Relative time (e.g., "2 hours ago")	System calculation	Display-only; Calculated from Last Modified Date

4.5 Validation Rules by Input Type

4.5.1 Text Input Fields

All text input fields shall adhere to the following rules:

Character Encoding:

- System shall accept UTF-8 encoded text
- All Unicode characters are permitted unless specified otherwise
- System shall preserve Unicode characters during save and retrieval

Whitespace Handling:

- Leading and trailing whitespace shall be automatically trimmed on save
- Multiple consecutive spaces shall be preserved within text
- Tab characters shall be converted to single spaces
- Line breaks shall be preserved in multi-line fields

Special Characters:

- HTML special characters (<, >, &, ", ') shall be escaped for display
- SQL injection characters shall be sanitized before database operations
- Script tags shall be stripped from all user input

4.5.3 Numeric Fields**Content Position Fields (DT-02, DT-03):**

- Shall accept only non-negative integers
- Shall validate that start position < end position
- Shall validate positions exist within document bounds
- Shall update automatically when document content changes

Usage Count (MD-06):

- Read-only field, user cannot edit
- Shall update in real-time when tags applied or removed
- Shall never be negative
- Shall display 0 for newly created tags

4.6 Validation Timing**Real-Time Validation (During Typing):**

- Character count updates on every keystroke
- Maximum length enforcement prevents typing beyond limit
- Format indicators (e.g., # prefix) appear automatically

On-Blur Validation (When Leaving Field):

- Required field check occurs when user tabs out
- Format validation occurs after user finishes typing

- Duplicate name check executes on blur
- Error messages appear below field with red text

On-Submit Validation (When Clicking Save/Create):

- All required fields checked for completion
- All format rules re-validated
- Cross-field validation (e.g., unique names) executed
- Form submission prevented if any validation fails
- Focus moves to first field with error

4.7 Error Message Standards

All error messages shall follow this format:

Structure: [Problem Statement]. [Required Action].

Examples:

- "Tag name is required. Please enter a tag name."
- "Tag name must be 1-50 characters. Current length: 52 characters."
- "A tag with this name already exists. Please choose a different name."

Error Message Display:

- **Position:** Directly below the related input field
- **Font:** 12px, Regular weight
- **Color:** #EA4335 (Error Red)
- **Icon:** Alert circle icon (12px) displayed to left of text
- **Animation:** Fade in over 150ms
- **Duration:** Remains visible until error is corrected
- **Dismissal:** Disappears when user corrects the error

Multiple Errors:

- If multiple fields have errors, all errors display simultaneously
- Focus moves to first field with error on submit
- User can navigate between fields to fix all errors
- Save/Create button remains disabled until all errors resolved

4.8 Data Sanitization

Before data is saved to the database, the system shall apply the following sanitization:

String Fields:

- Trim leading and trailing whitespace
- Remove null bytes (\0)
- Escape HTML special characters for display
- Limit to maximum length specified in validation rules
- Convert to UTF-8 encoding if not already

Tag and Project Names:

- Convert to lowercase for uniqueness comparison
- Store original case for display purposes
- Remove multiple consecutive spaces
- Strip special characters except allowed ones

Free Text Fields (Descriptions):

- Remove HTML tags and script elements
- Preserve line breaks and basic formatting
- Escape quotes and apostrophes for SQL safety
- Maintain Unicode character support

4.9 Data Constraints Summary

Table 6: Data Constraints Summary

Constraint Type	Rule	Applies To	Enforcement
Uniqueness	Tag names must be unique within project scope	TG-01	Database unique constraint + client-side check

Uniqueness	Project names must be unique system-wide	PJ-01	Database unique constraint + client-side check
Required Fields	Must have non-empty value	TG-01, TG-02, PJ-01, PJ-02	Form validation prevents submission
Length Limits	Maximum character count	All string fields	Input maxlength attribute + server validation
Format	Specific pattern match	TG-01, PJ-01	Regex validation on client and server
Referential Integrity	Must reference existing record	TG-03, DT-01	Foreign key constraints in database
Range	Numeric value within bounds	DT-02, DT-03	Comparison validation
Data Type	Must match specified type	All fields	TypeScript typing + database schema